

Data Analytics and CRISP/DM Methodology Final Project

Topic: Laptop Prices on the Global Market.

Name: Chris Doyle

Employee Number: 92885

Statement of Authentication:

By submitting this assignment, I consent that this work is my own, and where I have made use of the ideas/work of another individual, I have made appropriate acknowledgment to the original author of said work in the bibliography of this assignment.

All workings and tables can be found in the files attached with this submission.

Signed:

Chris Doyle

Final Project - Data Analytics and Visualization	4
Project Report Template	4
1. Introduction	4
Motivation for Report	4
Limitations	4
2. Data	5
3. Importing	6
Laptop Prices	6
PCPartPicker	7
CPU	7
4. Data Preparation	11
Data Types	11
Missing Values	12
Duplicate Rows:	13
Logical Integrity Issues	13
Outlier Values	14
Summary Statistics	15
5. Data Visualization	17
Univariate Analysis	17
Bivariate Analysis	19
6. Insights	20
1 Ram Has a Strong Correlation with Price	20
2 Popularity of Windows	21
3 Intel Is the Leading Producer of CPUs	23
4. Clustering of HDD, SSD, and Flash Storage	24
5. FullHD is More Popular Than Standard Definition	25
6. Workstation, Ultrabook and Gaming PCs are, on average, more expensive than Other Computers.	27
7. All 4 Datasets Had a Positive Skew and Multiple Outliers	28
8. The Vast Majority of Gaming Keyboards Had Built-In Lighting	29
7. Machine Learning	30
8. Conclusion	30
Bibliography:	31
Appendix 1: Graphical Analysis	33
Univariate	33
Laptop Prices	33
Headphones	41
Mouse	44
Keyboards	46

Bivariate	49
Laptop Prices	49
Appendix 2: Data Features	55
1. Laptop Prices Dataset Explanation	55
2. PCPartPicker API Dataset Explanation	57
CPU	57
Headphones	57
Mice	58
Keyboards	59
Appendix 3: Data Quality Issues and Remedies	60
Laptop Prices	60
Laptop Prices (New Features)	61
PCPartPicker API Dataset Explanation	62
Headphones	62
Mice	63
Keyboards	64
Appendix 4: Potential Price Gouging Example	65
Lexicon	66

Final Project - Data Analytics and Visualization

Project Report Template

1. Introduction

Motivation for Report

Fluctuations in the availability of graphics processor units (GPUs) over the last several years caused a significant increase in the price of high-end personal computers or “PCs” (Griffith, 2023). The shortage has been attributed to manufacturing woes, heightened demand due to the cryptocurrency mining, and the logistical challenges of the Coronavirus pandemic (Molloy, 2021).

Much of the media attention has been on GPUs, but during the preliminary research into this topic, little coverage was given to other parts such as CPUs. This report will perform an analysis of computer prices using the CRISP-DM model, identify features that influence the price of each device, and discuss the prospects of a machine learning model to model PC prices from the perspective of a new entrant into the PC market.

Limitations

The main dataset contains only information on laptops, as it was not possible to find a dataset that contained both desktop and Laptops. Samples were taken and compared to prices of reputable retailers to ensure the data was legitimate.

2. Data

Data for this project was sourced from Kaggle user Wagih (2024), a csv file listing a series of computers, their price, and their components referred to as *Laptop_prices.csv*. This data was supported using the PCPartPicker-API (Simon, 2024) to include prices of complementary goods. A comprehensive description of each feature is provided in Appendix 2.

The Laptop prices file is available [here](#).

The PCPartPicker-API documentation is available [here](#) and install instructions [here](#).

The PCPartPicker-API dataset is a python dictionary with 21 different keys, each key representing different types of parts. A full list of these parts is included as figure 2.1.

Figure 2.1: Parts featured in the PCPart Picker API

```
header("List of Parts")
for i in allData.keys():
    print(i)
```

```
List of Parts
keyboard
sound-card
external-hard-drive
mouse
thermal-paste
ups
case-fan
cpu-cooler
speakers
internal-hard-drive
power-supply
optical-drive
monitor
wired-network-card
video-card
wireless-network-card
case
motherboard
headphones
cpu
memory
fan-controller
```

3. Importing

Laptop Prices

The laptop prices dataset was taken from a microsoft excel comma separated values file. The file functions as a conventional CSV file and was imported using the `read_csv` method, referred to as `df`. (see figure 3.1.1).

The head method was used to check the first 8 rows to inspect that the data had successfully been imported (see figure 3.1.2).

Figure 3.1.1: Code to Import the *Laptop_prices.csv* File

```
# Get Laptop prices file  
  
df = pd.read_csv('laptop_prices.csv')
```

Figure 3.1.2: Initial Inspection of Laptop prices database

```
[68]: #Inspect first 8 rows  
df.head(8)
```

	Company	Product	TypeName	Inches	Ram	OS	Weight	Price_euros	Screen	ScreenW	...	RetinaDisplay	CPU_company	CPU_freq	CPU_model	PrimarySt
0	Apple	MacBook Pro	Ultrabook	13.3	8	macOS	1.37	1339.69	Standard	2560	...	Yes	Intel	2.3	Core i5	
1	Apple	Macbook Air	Ultrabook	13.3	8	macOS	1.34	898.94	Standard	1440	...	No	Intel	1.8	Core i5	
2	HP	250 G6	Notebook	15.6	8	No OS	1.86	575.00	Full HD	1920	...	No	Intel	2.5	Core i5 7200U	
3	Apple	MacBook Pro	Ultrabook	15.4	16	macOS	1.83	2537.45	Standard	2880	...	Yes	Intel	2.7	Core i7	
4	Apple	MacBook Pro	Ultrabook	13.3	8	macOS	1.37	1803.60	Standard	2560	...	Yes	Intel	3.1	Core i5	
5	Acer	Aspire 3	Notebook	15.6	4	Windows 10	2.10	400.00	Standard	1366	...	No	AMD	3.0	A9-Series 9420	
6	Apple	MacBook Pro	Ultrabook	15.4	16	Mac OS X	2.04	2139.97	Standard	2880	...	Yes	Intel	2.2	Core i7	
7	Apple	Macbook Air	Ultrabook	13.3	8	macOS	1.34	1158.70	Standard	1440	...	No	Intel	1.8	Core i5	

The row counts and column counts were inspected (see figure 3.1.3). The row and column counts are 1275 and 23 respectively. The row count was deemed sufficient, as it would allow for a 70:30 training/test split with ample data for both.

Figure 3.1.3: Initial Inspection of Row Count and Column Count

```
[63]: # Get shape of dataset
print("Row Count: ", df.shape[0])
print("Col Count: ", df.shape[1])

Row Count: 1275
Col Count: 23
```

PCPartPicker

The various part datasets were retrieved using the PCPartPicker API. After installing the PCPartPicker package using pip (see figure 3.2.0).

Figure 3.2.0: Code Used to Import the Entire PCPartPicker Catalog as a Python Dictionary*

```
# Apply Nest Asyncio
nest_asyncio.apply()

# Note: without this piece of code, the API program will fail, as it detects that there is another program running on this computer (Jupyter)
# In order to prevent throwing up an error, the line of code above enables the api to believe there isn't any other program running but it

# Pull the entire database of parts
api = pcpartpicker.API()
allData = api.retrieve_all()
```

*import *PCPartPicker* run earlier in the script.

CPU

The PC PartPicker dataset was composed of multiple different datasets. Each part is stored in a class based on the nature of the part. For example, figure 3.2.1 demonstrates the CPU parts. This class was converted to a pandas dataframe (see figure 3.2.2).

Figure 3.2.1: Initial Inspection of CPU Class.

```
[266]: # Inspect the CPU data
cpuData = allData['cpu']
cpuData

[266]: [CPU(brand='AMD', model='2650', cores=2, base_clock=ClockSpeed(cycles=1450000000), boost_clock=None, tdp=25, integrated_graphics='Radeon HD 8240', multithreading=False, price=<Money: 59.61 USD>),
CPU(brand='AMD', model='3850', cores=4, base_clock=ClockSpeed(cycles=1300000000), boost_clock=None, tdp=25, integrated_graphics='Radeon HD 8280', multithreading=False, price=<Money: 79.59 USD>),
CPU(brand='AMD', model='5150', cores=4, base_clock=ClockSpeed(cycles=1600000000), boost_clock=None, tdp=25, integrated_graphics='Radeon HD 8400', multithreading=False, price=<Money: 0.00 USD>),
CPU(brand='AMD', model='5350', cores=4, base_clock=ClockSpeed(cycles=2049999999), boost_clock=None, tdp=25, integrated_graphics='Radeon HD 8400', multithreading=False, price=<Money: 104.94 USD>),
CPU(brand='AMD', model='5350', cores=4, base_clock=ClockSpeed(cycles=2049999999), boost_clock=None, tdp=25, integrated_graphics='Radeon HD 8400', multithreading=False, price=<Money: 0.00 USD>),
CPU(brand='AMD', model='A10-5700', cores=4, base_clock=ClockSpeed(cycles=3400000000), boost_clock=None, tdp=65, integrated_graphics='Radeon HD 7660D', multithreading=False, price=<Money: 0.00 USD>),
CPU(brand='AMD', model='A10-5800K', cores=4, base_clock=ClockSpeed(cycles=3800000000), boost_clock=None, tdp=100, integrated_graphics='Radeon HD 7660D', multithreading=False, price=<Money: 0.00 USD>),
CPU(brand='AMD', model='A10-6700', cores=4, base_clock=ClockSpeed(cycles=3700000000), boost_clock=None, tdp=65, integrated_graphics='Radeon HD 8670D', multithreading=False, price=<Money: 0.00 USD>),
CPU(brand='AMD', model='A10-6700T', cores=4, base_clock=ClockSpeed(cycles=2500000000), boost_clock=ClockSpeed(cycles=3500000000), tdp=45, integrated_graphics='Radeon HD 8650D', multithreading=False, price=<Money: 0.00 USD>)]
```

Figure 3.2.2: Conversion of CPU Parts From Class Type CPU to Dataframe

```
# Initial Dictionary
cpuDataDictionary = {
    "brand" : [cpu.brand for cpu in cpuData],
    "model" : [cpu.model for cpu in cpuData],
    "cores" : [cpu.cores for cpu in cpuData],
    "base_clock_speed": [cpu.base_clock.cycles for cpu in cpuData],
    "tdp" : [cpu.tdp for cpu in cpuData],
    "integrated_graphics" : [cpu.integrated_graphics for cpu in cpuData],
    "multithreading" : [cpu.multithreading for cpu in cpuData],
    "price" : [cpu.price for cpu in cpuData]
}

#Dictionary to dataframe
dfCPU = pd.DataFrame(cpuDataDictionary)
dfCPU
```

Figure 3.2.2: Conversion of CPU Parts From Class Type CPU to Dataframe (Continued)

	brand	model	cores	base_clock_speed(cycles)	tdp	integrated_graphics	multithreading	price
0	AMD	2650	2	1450000000	25	Radeon HD 8240	False	US\$59.61
1	AMD	3850	4	1300000000	25	Radeon HD 8280	False	US\$79.59
2	AMD	5150	4	1600000000	25	Radeon HD 8400	False	US\$0.00
3	AMD	5350	4	2049999999	25	Radeon HD 8400	False	US\$104.94
4	AMD	5350	4	2049999999	25	Radeon HD 8400	False	US\$0.00
...	...	...	...	...	...	...	...	...
1066	Intel	Xeon E5-2698 V4	20	2200000000	135	None	True	US\$0.00
1067	Intel	Xeon E5-2699 V3	18	2300000000	145	None	True	US\$3,089.95
1068	Intel	Xeon E5-2699 V4	22	2200000000	145	None	True	US\$1,519.93
1069	Intel	Xeon E5-4620	8	2200000000	95	None	True	US\$1,599.99
1070	Intel	Xeon E5-4650	8	2700000000	130	None	True	US\$948.89

It was not possible to join the CPU data (dfCPU) with the laptop data (df) and include the CPU data as the common columns used to join the two tables (CPU_model on df and model on dfCPU) did not share any features, as there were no CPUs in the parts picker dataset that existed in the laptops dataset and vice versa. (see figure 3.2.3).

Figure 3.2.3: Inspection for Common Values Between CPU and Laptop Dataset

```
# Inspect the cpu model columns of both tables to see there is crossover
df[df["CPU_model"].isin(dfCPU["model"])]
```

Company	Product	TypeName	Inches	Ram	OS	Weight	Price_euros	Screen
---------	---------	----------	--------	-----	----	--------	-------------	--------

The logic from figure 3.2.2 was implemented to analyse complementary products that could be marketed alongside potential products, such as headphones (figure 3.2.4), mice (see figure 3.2.5), and keyboards (see figure 3.2.6). The dimensions of the datasets are included below in figure 3.2.7.

Figure 3.2.4: Conversion of Headphones Parts From the Headphones Class to Dataframe

```
#Headphones
#Convert the dataset from a series of objects in Python (CPU class) to a dictionary, and then from a dictionary to a Dataframe
hpData = allData['headphones']

# Inital Dictionary
headphonesDataDictionary = {
    "brand" : [obj.brand for obj in hpData],
    "model" : [obj.model for obj in hpData],
    "form_factor" : [obj.form_factor for obj in hpData],
    "frequency_response": [obj.frequency_response for obj in hpData],
    "has_microphone" : [obj.has_microphone for obj in hpData],
    "is_wireless" : [obj.is_wireless for obj in hpData],
    "color" : [obj.color for obj in hpData],
    "price" : [obj.price for obj in hpData]
}
```

Figure 3.2.5: Conversion of Mouse Parts From the Mouse Class to Dataframe

```
#Mouses
#Convert the dataset from a series of objects in Python (CPU class) to a dictionary, and then from a dictionary to a Dataframe
mouseData = allData['mouse']

# Inital Dictionary
mouseDataDictionary = {
    "brand" : [obj.brand for obj in mouseData],
    "model" : [obj.model for obj in mouseData],
    "tracking" : [obj.tracking for obj in mouseData],
    "connection": [obj.connection for obj in mouseData],
    "max_dpi" : [obj.max_dpi for obj in mouseData],
    "hand_orientation" : [obj.hand_orientation for obj in mouseData],
    "color" : [obj.color for obj in mouseData],
    "price" : [obj.price for obj in mouseData]
}

#Dictionary to dataframe
dfMouse = pd.DataFrame(mouseDataDictionary)
dfMouse
```

Figure 3.2.6: Conversion of CPU Parts From the CPU Class to Dataframe (Continue)

```
#Keyboards
#Convert the dataset from a series of objects in Python (CPU class) to a dictionary, and then from a dictionary to a Dataframe
KeyboardData = allData['keyboard']

# Inital Dictionary
KeyboardDataDictionary = {
    "brand" : [obj.brand for obj in KeyboardData],
    "model" : [obj.model for obj in KeyboardData],
    "style" : [obj.style for obj in KeyboardData],
    "switches": [obj.switches for obj in KeyboardData],
    "backlight" : [obj.backlight for obj in KeyboardData],
    "tenkeyless" : [obj.tenkeyless for obj in KeyboardData],
    "connection" : [obj.connection for obj in KeyboardData],
    "color" : [obj.color for obj in KeyboardData],
    "price" : [obj.price for obj in KeyboardData]
}
```

Figure 3.2.7: Size and Shape of Each PCPartsPicker Dataset

Details	Headphones	Mice	Keyboards
Row Count	2417	2016	2342
Column Count	9	8	9

Code used to generate these figures:

```
def get_shape(df):  
    # get the number of columns and rows in the dataset:  
    print("Row Count: ", df.shape[0])  
    print("Col Count: ", df.shape[1])
```

4. Data Preparation

Data Types

Each column in the four datasets were checked to determine if they were using the optimal data type. The data types were reviewed (see figure 4.1). Boolean columns (i.e. columns with values yes or no) were converted to binary values (False for no and True for yes) to provide a standardised approach to columns of this nature.

Additionally, the price columns in the PCPartsPicker datasets were not correctly stored as floats but using the moneyed class (Wu, 2021) and were converted to floating points to simplify analysis (see figure 4.2 for code). They were also stored as USD, while the Laptop Prices Table used euros. The US prices were converted to Euros using the present exchange rate of \$1=€0.9438 (Financial Times, 2024).

Figure 4.1: Data Type Inspection of Laptop Prices Dataset.

```
[65]: #Get Data Types
      df.dtypes

[65]: Company          object
      Product          object
      TypeName         object
      Inches          float64
      Ram              int64
      OS              object
      Weight          float64
      Price_euros      float64
      Screen           object
      ScreenW          int64
      ScreenH          int64
      Touchscreen      object
      IPSpanel         object
      RetinaDisplay     object
      CPU_company      object
      CPU_freq         float64
      CPU_model        object
      PrimaryStorage    int64
      SecondaryStorage  int64
      PrimaryStorageType object
      SecondaryStorageType object
      GPU_company      object
      GPU_model        object
      dtype: object
```

Figure 4.2: Converting Price from Moneyed Class to Floating Point

```
dfMouse['priceAdj'] = dfMouse["price"].apply(lambda x: x.amount).astype(float)
dfMouse['priceAdj']
```

Missing Values

The datasets were inspected for missing values using the logic featured in figure 4.3 below, no missing values were observed within the laptop prices dataset. However, many of the PC parts data had missing values, which were replaced with “not available” for categorical columns (see figure 4.4 for code).

Figure 4.3: Code to Inspect for Missing Values

```
# Check For Missing Values
missing = []
for c in df.columns.to_list():
    missing.append(df.isna().sum())
print(missing)
```

[Company	0
Product	0
TypeName	0
Inches	0
Ram	0
OS	0
Weight	0
Price_euros	0
Screen	0
ScreenW	0
ScreenH	0
Touchscreen	0
IPSPanel	0
RetinaDisplay	0
CPU_company	0
CPU_freq	0
CPU_model	0
PrimaryStorage	0

Figure 4.4: Missing Rows from PCPartsPicker Headphone Dataset Example

```
# Check For Missing Values
inspect_missing_values(dfHeadphones)
```

[brand	0
model	0
form_factor	0
frequency_response	558
has_microphone	0
is_wireless	0
color	9
price	0
dtype: int64]	

```
#Check what % missing
inspect_missing_values_percentage(dfHeadphones)
```

brand	0.000000
model	0.000000
form_factor	0.000000
frequency_response	23.086471
has_microphone	0.000000
is_wireless	0.000000
color	0.372362
price	0.000000
dtype: float64	

```
# Replace missing values with not available
dfHeadphones["frequency_response"].fillna("not available")
```

Additionally, all PCPartPicker datasets had a substantial number of rows missing price information due to data collection issues (see Appendix 3 for further details). These rows were discarded as they were missing the primary target feature for each dataset.

Duplicate Rows:

The datasets were inspected for duplicate rows using the code demonstrated in figure 4.5. No duplicate rows were identified.

Figure 4.5: Code For Checking Duplicate Rows

```
def check_duplicates(df):  
    #return the number of duplicate rows in the dataset  
    dfDuplicateRows = df.duplicated().sum()  
    return dfDuplicateRows
```

Logical Integrity Issues

The datasets were checked for logical integrity issues, such as rows having a negative price, but no such rows occurred in any dataset (see figure 4.6 & 4.8). The only logical integrity issue observed was with the primary and secondary storage features, as these columns treat 1 GB of storage of SSD, HDD and Flash storage as identical, when in reality SSD drives provide faster response times and improved performance (Nelson, 2024). The rows were converted into four new features as seen in figure 4.8 to provide a more detailed breakdown of consumer storage requirements.

Figure 4.6: Check for Negative Prices

```
#Check remaining Columns  
header("Check Negative Values")  
columns = ["PrimaryStorage", "SecondaryStorage", "StorageSSD", "StorageFlash", "StorageHDD", "StorageHybrid"]  
  
for i in columns:  
    print(i, ":", check_negative_values(df, i).shape[0])  
  
Check Negative Values  
PrimaryStorage : 0  
SecondaryStorage : 0  
StorageSSD : 0  
StorageFlash : 0  
StorageHDD : 0  
StorageHybrid : 0
```

Figure 4.7 Logic for check_negative_values Function Featured in Figure 4.6:

```
def check_negative_values(df, column):  
    result = df[df[column] < 0]  
    return result
```

Figure 4.8: Conversion of PrimaryStorage and Secondary Storage to StorageSSD, StorageFlash, StorageHDD, StorageHybrid

```
# Create SSD storage Feature
df["StorageSSD"] = 0
df.loc[df["PrimaryStorageType"] == "SSD", "StorageSSD"] += df["PrimaryStorage"]
df.loc[df["SecondaryStorageType"] == "SSD", "StorageSSD"] += df["SecondaryStorage"]

# Create Flash Storage Feature
df["StorageFlash"] = 0
df.loc[df["PrimaryStorageType"] == "Flash Storage", "StorageFlash"] += df["PrimaryStorage"]
df.loc[df["SecondaryStorageType"] == "Flash Storage", "StorageFlash"] += df["SecondaryStorage"]

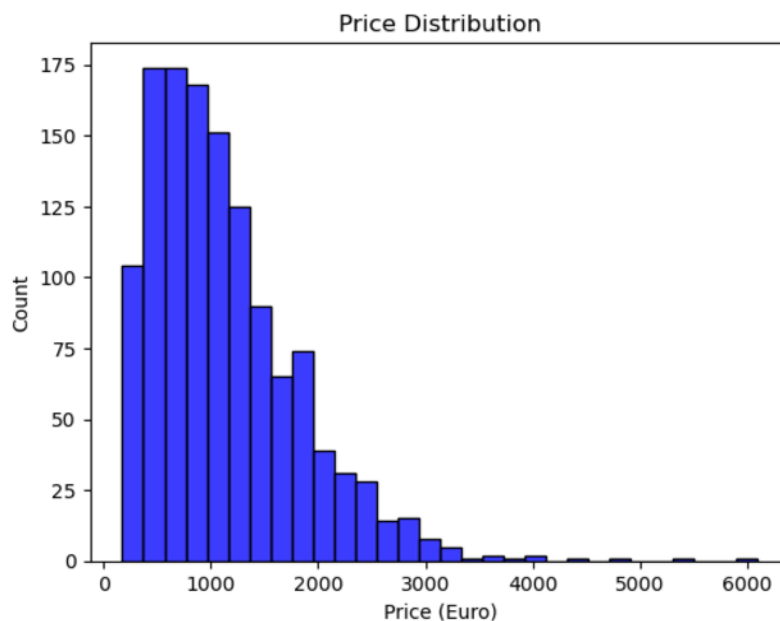
# Create HDD storage Feature
df["StorageHDD"] = 0
df.loc[df["PrimaryStorageType"] == "HDD", "StorageHDD"] += df["PrimaryStorage"]
df.loc[df["SecondaryStorageType"] == "HDD", "StorageHDD"] += df["SecondaryStorage"]

# Create StorageHybrid Feature
df["StorageHybrid"] = 0
df.loc[df["PrimaryStorageType"] == "Hybrid", "StorageHybrid"] += df["PrimaryStorage"]
df.loc[df["SecondaryStorageType"] == "Hybrid", "StorageHybrid"] += df["SecondaryStorage"]
```

Outlier Values

Due to the high cardinality of the Price feature and its importance as the target feature, the price feature was checked for outliers, visible on Figure 4.9.

Figure 4.9: Price Distribution of Laptop Prices (Before Outliers Removed)



There are some outlier values for prices outside the €4000-6000 range. A quick google search of the outliers revealed that these prices were overstated for the models in question, and the outliers were removed from the dataset. Similar corrections were applied to the other datasets (see Appendix 3).

Summary Statistics

The main dataset was split into a categorical dataset and a numeric dataset to simplify the calculation of summary statistics (the logic for doing so can be viewed in figure 4.10 & 4.11). The feature TotalStorage was also created to capture the total storage that computers were utilizing (see figure 4.12).

Figure 4.10: Separation of Laptop Prices Dataset into Categorical and Continuous Versions

```
[ ]: # The Laptop prices data will be seperated into Numeric and Categorical to assess summary statistics
# issues exist and what corrections must be made to address these issues
dfSplit = split_data(df)

dfContinuous = dfSplit['numeric']
dfCategorical = dfSplit['categorical']
```

Figure 4.11: Logic for split_data Function:

```
def split_data(df, removeNulls=False):
    #split the data into numeric and categorical

    #splits the columns between categorial and continous data:
    dfContinuousColumns = df.select_dtypes(['float64','int64']).columns
    dfCategoricalColumns = df.select_dtypes(['object','bool']).columns

    #returns the two datasets in a dictionary
    results = { "numeric": df[dfContinuousColumns],
                "categorical": df[dfCategoricalColumns]
              }

    return results
```

Figure 4.12: TotalStorage Feature

```
[ ]: # Get Total Storage Feature
df["TotalStorage"] = df["PrimaryStorage"] + df["SecondaryStorage"]
```

The code used to compute these figures has been included in figure 4.13 & 4.14. A full list of the DQ issues observed and the actions taken to clean this data has been included in Appendix 3.

Figure 4.13: Calculation of Summary Statistics (Categorical)

```
def get_categorical_details(dfCategorical):
    #return summary statistics on a series of categorial features
    #get count and cardinality
    dfCategoricalSummary = dfCategorical.describe().T

    # Get the mode of each categorical feature feature
    category_modes = pd.DataFrame(index=dfCategoricalSummary.index, columns=['mode', 'freq_mode', 'freq_mode(%)', '2ndmode', 'freq_2ndmode', 'freq_2ndmode(%)'])

    for i in dfCategorical.columns.to_list():
        #get mode data
        category_modes.loc[i, 'mode'] = dfCategorical[i].value_counts().keys()[0]
        category_modes.loc[i, 'freq_mode'] = dfCategorical[i].value_counts().loc[category_modes.loc[i, 'mode']]
        category_modes.loc[i, 'freq_mode(%)'] = dfCategorical[i].value_counts(normalize=True).loc[category_modes.loc[i, 'mode']]

        #get secondary mode data
        category_modes.loc[i, '2ndmode'] = dfCategorical[i].value_counts().keys()[1]
        category_modes.loc[i, 'freq_2ndmode'] = dfCategorical[i].value_counts().loc[category_modes.loc[i, '2ndmode']]
        category_modes.loc[i, 'freq_2ndmode(%)'] = dfCategorical[i].value_counts(normalize=True).loc[category_modes.loc[i, '2ndmode']]

    #merge two datasets together
    dfCategoricalSummary = pd.concat([dfCategoricalSummary[["count", "unique"]], category_modes], axis = 1)
    return dfCategoricalSummary
```

Figure 4.14: Calculation of Summary Statistics (Numeric)

```
def get_continuous_details(dfContinuous):
    #return summary statistics on the continous data features
    #get main summary statistics
    dfContinuousSummary = dfContinuous.describe().T

    #Get Cardinality
    dfContinuousCardinality = pd.DataFrame({"Unique": dfContinuous.nunique()})

    #merge data into single dataframe
    dfContinuousSummary = pd.concat([dfContinuousSummary, dfContinuousCardinality], axis = 1)

    return dfContinuousSummary
```


5. Data Visualization

Note: Insights from these statistics will be discussed in section 6. All graphs for this section are included in Appendix 1.

Univariate Analysis

The summary statistics generated as part of section four have been included in figures below.

Figure 5.1.1: Summary Statistics for Laptop Prices Dataset (Categorical Features)

	count	unique	mode	freq_mode	freq_mode(%)	2ndmode	freq_2ndmode	freq_2ndmode(%)
Company	1271	19	Dell	291	0.228954	Lenovo	288	0.226593
Product	1271	617	XPS 13	30	0.023603	Inspiron 3567	25	0.01967
TypeName	1271	6	Notebook	706	0.555468	Gaming	203	0.159717
OS	1271	9	Windows 10	1045	0.822187	No OS	66	0.051928
Screen	1271	4	Full HD	834	0.656176	Standard	369	0.290323
Touchscreen	1271	2	False	1085	0.853659	True	186	0.146341
IPSPanel	1271	2	False	916	0.720692	True	355	0.279308
RetinaDisplay	1271	2	False	1254	0.986625	True	17	0.013375
CPU_company	1271	3	Intel	1210	0.952006	AMD	60	0.047207
CPU_model	1271	91	Core i5 7200U	193	0.151849	Core i7 7700HQ	147	0.115657
PrimaryStorageType	1271	4	SSD	833	0.655389	HDD	359	0.282455
SecondaryStorageType	1271	4	No	1063	0.836349	HDD	202	0.15893
GPU_company	1271	4	Intel	704	0.553895	Nvidia	392	0.308419
GPU_model	1271	110	HD Graphics 620	279	0.219512	HD Graphics 520	181	0.142408

Figure 5.1.2: Summary Statistics for Laptop Prices Dataset (Numeric Features)

	count	mean	std	min	25%	50%	75%	max	Unique
Inches	1271.0	15.017073	1.427331	10.10	14.0	15.60	15.6	18.4	18
Ram	1271.0	8.379229	4.970877	2.00	4.0	8.00	8.0	64.0	9
Weight	1271.0	2.037128	0.667100	0.69	1.5	2.04	2.3	4.7	171
Price_euros	1271.0	1122.108222	662.225815	174.00	606.0	985.00	1491.4	3975.0	787
ScreenW	1271.0	1895.449253	485.021248	1366.00	1920.0	1920.00	1920.0	3840.0	13
ScreenH	1271.0	1071.335956	279.375771	768.00	1080.0	1080.00	1080.0	2160.0	10
CPU_freq	1271.0	2.300944	0.503303	0.90	2.0	2.50	2.7	3.6	25
PrimaryStorage	1271.0	443.701023	365.345888	8.00	256.0	256.00	512.0	2048.0	13
SecondaryStorage	1271.0	176.623131	416.497630	0.00	0.0	0.00	0.0	2048.0	6
StorageSSD	1271.0	184.632573	184.952864	0.00	0.0	256.00	256.0	1024.0	12
StorageFlash	1271.0	4.519276	30.535294	0.00	0.0	0.00	0.0	512.0	7
StorageHDD	1271.0	423.521637	530.328626	0.00	0.0	0.00	1024.0	2048.0	6
StorageHybrid	1271.0	7.650669	87.037203	0.00	0.0	0.00	0.0	1024.0	3
TotalStorage	1271.0	620.324154	482.438930	8.00	256.0	500.00	1024.0	2560.0	22

Figure 5.1.3: Summary Statistics for PCPartsPicker Headphones Dataset (Categorical Features)

	count	unique	mode	freq_mode	freq_mode(%)	2ndmode	freq_2ndmode	freq_2ndmode(%)
brand	722	85	Sennheiser	73	0.101108	Audio-Technica	54	0.074792
model	722	682	Gaming A10	3	0.004155	F.R.E.Q. 3	3	0.004155
form_factor	722	4	Circumaural	415	0.574792	Supra-aural	132	0.182825
has_microphone	722	2	True	409	0.566482	False	313	0.433518
is_wireless	722	2	False	620	0.858726	True	102	0.141274
color	722	50	Black	221	0.306094	Black / Silver	144	0.199446

Figure 5.1.4: Summary Statistics for PCPartsPicker Headphones Dataset (Numeric Features)

	count	mean	std	min	25%	50%	75%	max	Unique
price	722.0	98.355686	93.443499	4.709562	28.620735	66.01881	141.52045	471.85281	436
frequency_response_min	722.0	15.422438	20.020383	0.000000	6.000000	18.00000	20.00000	300.00000	28
frequency_response_max	722.0	19420.775623	10747.898441	0.000000	20000.000000	20000.00000	24000.00000	50000.00000	38

Figure 5.1.5: Summary Statistics for PCPartsPicker Mice Dataset (Categorical Features)

	count	unique	mode	freq_mode	freq_mode(%)	2ndmode	freq_2ndmode	freq_2ndmode(%)
brand	759	70	Logitech	153	0.201581	Microsoft	84	0.110672
model	759	664	M325	18	0.023715	M310	8	0.01054
tracking	759	4	Optical	600	0.790514	Laser	144	0.189723
connection	759	3	Wired	379	0.499341	Wireless	335	0.44137
max_dpi	759	45	not available	264	0.347826	1000.0	110	0.144928
hand_orientation	759	4	Both	389	0.512516	Right	325	0.428195
color	759	58	Black	350	0.461133	Black / Silver	43	0.056653

Figure 5.1.6: Summary Statistics for PCPartsPicker mice Dataset (Numeric Features)

	count	mean	std	min	25%	50%	75%	max	Unique
price	759.0	45.861194	36.896913	4.709562	18.300282	33.778602	65.754546	197.2542	422

Figure 5.1.7: Summary Statistics for PCPartsPicker Keyboards Dataset (Categorical Features)

	count	unique	mode	freq_mode	freq_mode(%)	2ndmode	freq_2ndmode	freq_2ndmode(%)
brand	2331	168	Ducky	262	0.112398	Logitech	139	0.059631
model	2331	1877	Keyboard	11	0.004719	KBT PURE	10	0.00429
style	2331	5	Gaming	840	0.36036	Standard	831	0.356499
switches	2331	91	not available	1211	0.51952	Cherry MX Brown	200	0.0858
backlight	2331	12	not available	1306	0.560275	RGB	401	0.172029
tenkeyless	2331	2	False	1774	0.761047	True	557	0.238953
connection	2331	3	Wired	1906	0.817675	Wireless	329	0.141141
color	2331	41	Black	1832	0.785929	White	114	0.048906

Figure 5.1.8: Summary Statistics for PCPartsPicker Keyboards Dataset (Numeric Features)

	count	mean	std	min	25%	50%	75%	max	Unique
price	706.0	86.937709	67.860762	7.069062	36.145181	66.047124	121.7502	396.386562	412

Additionally, value counts were created using Matplotlib for each feature, and Seaborn histograms & Boxplots were used to capture the price distributions of all 4 datasets. These figures (representing all graphs 5.1.9 and above) have been included in Appendix 1.

Bivariate Analysis

As part of the Bivariate Analysis, the correlation coefficient of each numeric feature in the Laptop Prices dataset was calculated. This will be relevant for section 6. All figures have been included in Appendix 1.

Figure 5.2.1: Correlation Coefficient of Numeric Data

	Inches	Ram	Weight	Price_euros	ScreenW	ScreenH	CPU_freq	PrimaryStorage	SecondaryStorage	StorageSSD	StorageFlash	StorageHDD	StorageHybrid	TotalStorage
Inches	1.00	0.23	0.83	0.05	-0.08	-0.11	0.30	0.26	0.39	-0.11	-0.23	0.53	0.05	0.54
Ram	0.23	1.00	0.38	0.73	0.40	0.39	0.36	0.00	0.41	0.59	-0.06	0.11	0.05	0.35
Weight	0.83	0.38	1.00	0.19	-0.05	-0.07	0.31	0.17	0.49	-0.07	-0.18	0.52	0.11	0.55
Price_euros	0.05	0.73	0.19	1.00	0.53	0.53	0.43	-0.15	0.32	0.66	-0.04	-0.09	0.03	0.16
ScreenW	-0.08	0.40	-0.05	0.53	1.00	0.99	0.17	-0.09	0.15	0.50	-0.01	-0.11	0.00	0.07
ScreenH	-0.11	0.39	-0.07	0.53	0.99	1.00	0.16	-0.09	0.14	0.50	0.02	-0.13	0.00	0.05
CPU_freq	0.30	0.36	0.31	0.43	0.17	0.16	1.00	0.05	0.23	0.26	-0.22	0.13	0.05	0.24
PrimaryStorage	0.26	0.00	0.17	-0.15	-0.09	-0.09	0.05	1.00	-0.24	-0.21	-0.09	0.56	0.10	0.55
SecondaryStorage	0.39	0.41	0.49	0.32	0.15	0.14	0.23	-0.24	1.00	0.11	-0.06	0.58	0.01	0.68
StorageSSD	-0.11	0.59	-0.07	0.66	0.50	0.50	0.26	-0.21	0.11	1.00	-0.15	-0.40	-0.05	-0.07
StorageFlash	-0.23	-0.06	-0.18	-0.04	-0.01	0.02	-0.22	-0.09	-0.06	-0.15	1.00	-0.12	-0.01	-0.12
StorageHDD	0.53	0.11	0.52	-0.09	-0.11	-0.13	0.13	0.56	0.58	-0.40	-0.12	1.00	-0.07	0.93
StorageHybrid	0.05	0.05	0.11	0.03	0.00	0.00	0.05	0.10	0.01	-0.05	-0.01	-0.07	1.00	0.08
TotalStorage	0.54	0.35	0.55	0.16	0.07	0.05	0.24	0.55	0.68	-0.07	-0.12	0.93	0.08	1.00

6. Insights

1 Ram Has a Strong Correlation with Price

Of all numeric features within the dataset, Ram has the highest correlation coefficient with Price, with a correlation Coefficient of 0.726 to the target feature of the dataset, Price (as seen in figures 6.1.1 and 6.1.2). This is followed by SSD storage (0.66), and screen width & height (0.534 & 0.531 respectively).

This highlights the significance of Ram in the price matrix, and represents a critical factor when designing new PCs for the market.

Figure 6.1.1: Heatmap of Correlation Coefficients from Figure 5.2.1 (made with Seaborn)

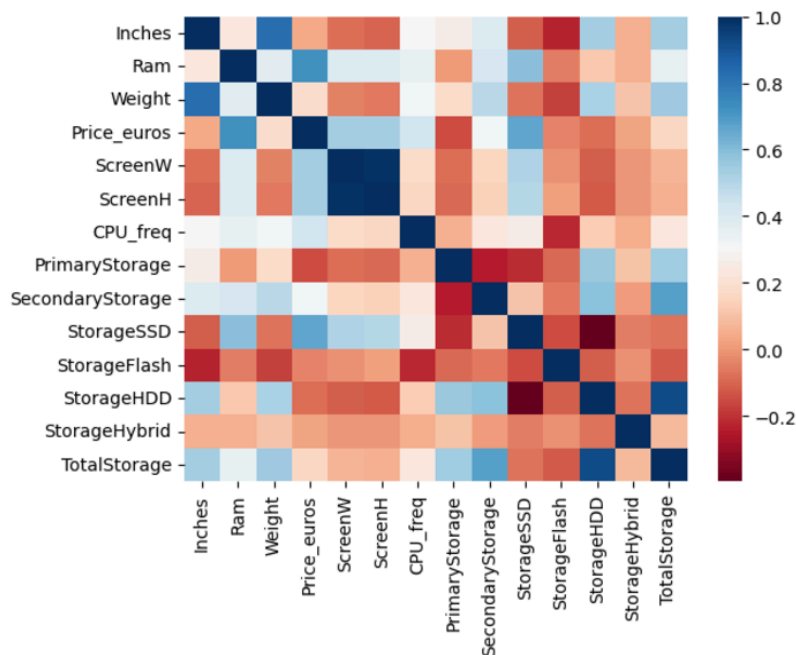


Figure 6.1.2: Correlation Coefficient of Price to Numeric Features

```
Inches      0.045055
Ram         0.725986
Weight      0.192778
Price_euros 1.000000
ScreenW     0.533934
ScreenH     0.530885
CPU_freq    0.430805
PrimaryStorage -0.147986
SecondaryStorage 0.316963
StorageSSD  0.664711
StorageFlash -0.036372
StorageHDD  -0.086961
StorageHybrid 0.025696
TotalStorage 0.161571
Name: Price_euros, dtype: float64
```

2 Popularity of Windows

Over 82% of Laptops surveyed used the Windows 10 operating system (see figure 6.2.1 & 6.2.2), with 3.46% using Windows 7 and 0.62% using Windows 10. However, when conducting a box plot analysis of the Price relative to OS, the OS price is only marginally above the market average (€1039 vs €985 respectively, see figure 6.2.3 & 6.2.4).

Figure 6.2.1: OS Breakdown

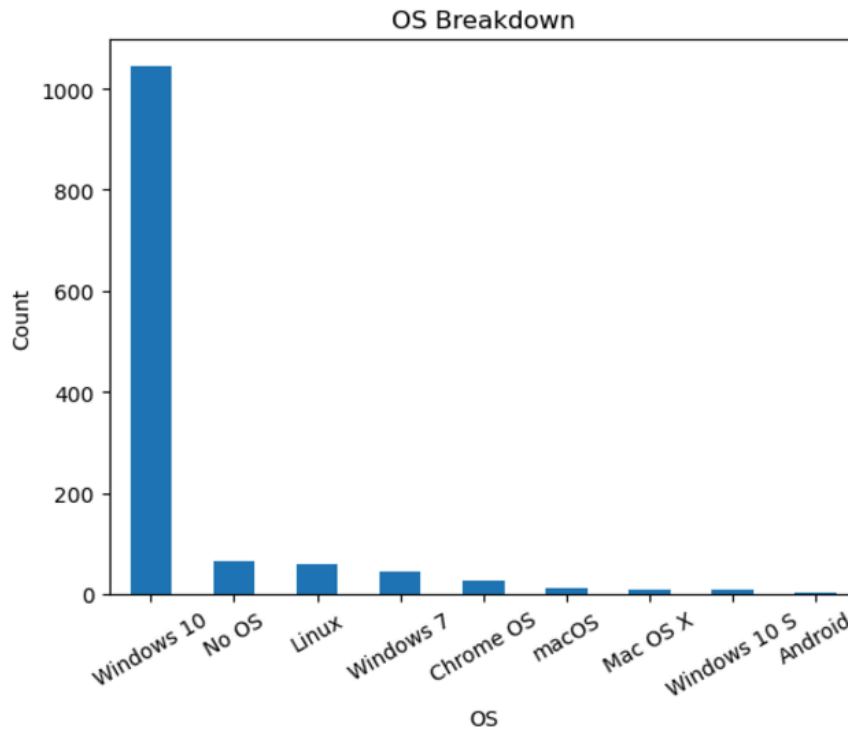


Figure 6.2.2: OS Breakdown (%)

```
: #Show % breakdown
(df['OS'].value_counts()/df['OS'].shape[0]) * 100

: OS
Windows 10      82.218725
No OS           5.192762
Linux           4.563336
Windows 7       3.461841
Chrome OS       2.124312
macOS           1.022817
Mac OS X        0.629426
Windows 10 S    0.629426
Android         0.157356
Name: count, dtype: float64
```

Figure 6.2.3: Boxplot Showing Price Relative to OS

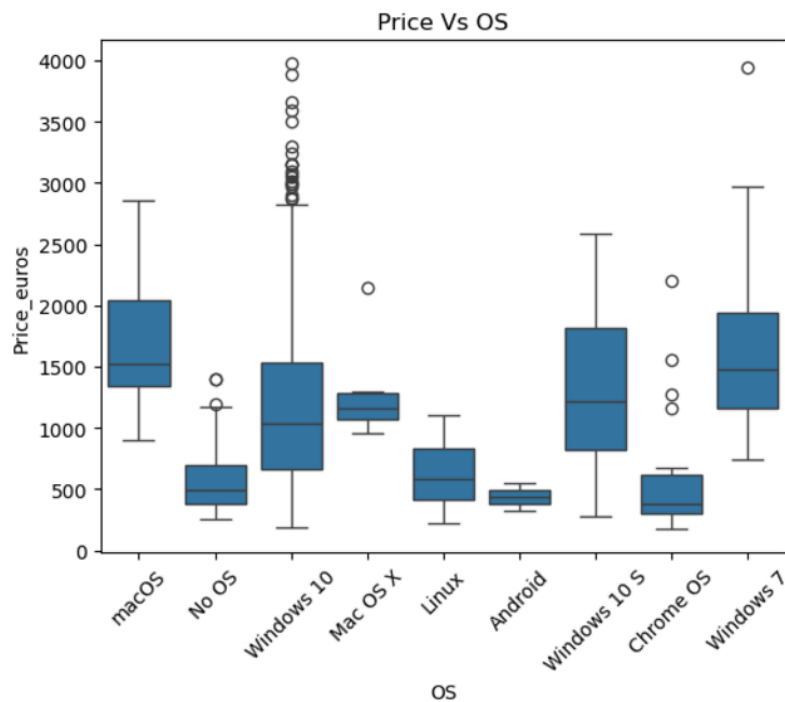


Figure 6.2.4: Summary Statistics for Price (Group Average vs Grouped by OS)

```
[571]: #Get summary for statistics for Prices
get_continuous_details(dfNumeric[["Price_euros"]])
```

	count	mean	std	min	25%	50%	75%	max	Unique
Price_euros	1271.0	1122.108222	662.225815	174.0	606.0	985.0	1491.4	3975.0	787

```
[573]: #Get summary for statistics for Prices grouped by OS
boxplot_statistics('OS')
```

	Mean	Std	Min	25%	50% (Median)	75%	Max
macOS	1749.630769	588.443390	898.94	1339.690	1518.55	2040.0000	2858.00
No OS	587.973333	277.151691	252.36	375.225	499.00	694.7500	1399.95
Windows 10	1168.286986	658.031020	191.90	659.000	1039.00	1534.0000	3975.00
Mac OS X	1262.871250	374.135623	959.00	1073.750	1164.00	1284.2500	2139.97
Linux	621.867069	251.586973	224.00	412.750	579.26	835.9975	1099.00
Android	434.000000	162.634560	319.00	376.500	434.00	491.5000	549.00
Windows 10 S	1286.480000	787.442491	277.99	826.750	1214.50	1816.2125	2589.00
Chrome OS	553.585185	470.057049	174.00	296.000	375.00	615.0000	2199.00
Windows 7	1625.234318	667.644290	737.00	1159.015	1482.00	1936.8450	3949.40

This highlights the popularity of Windows among the general consumer base, owing to its user-friendly interface (Short, 2020, pp. 10). This should be a consideration for any businesses wishing to break into the PC market when determining what OS to implement into their product.

3 Intel Is the Leading Producer of CPUs

As demonstrated in figure 6.3.1, Intel is responsible for producing CPUs for over 95% of all Laptops surveyed. This is not the case for the GPU market, where manufacturers have a more normalised market share (see figure 6.3.2).

Figure 6.3.1: CPU Manufacturer Analysis

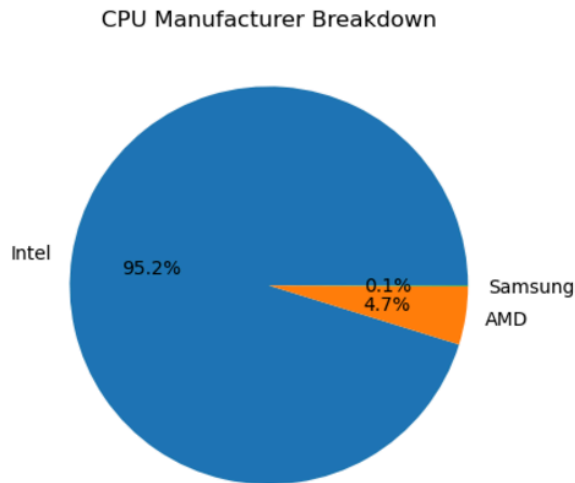
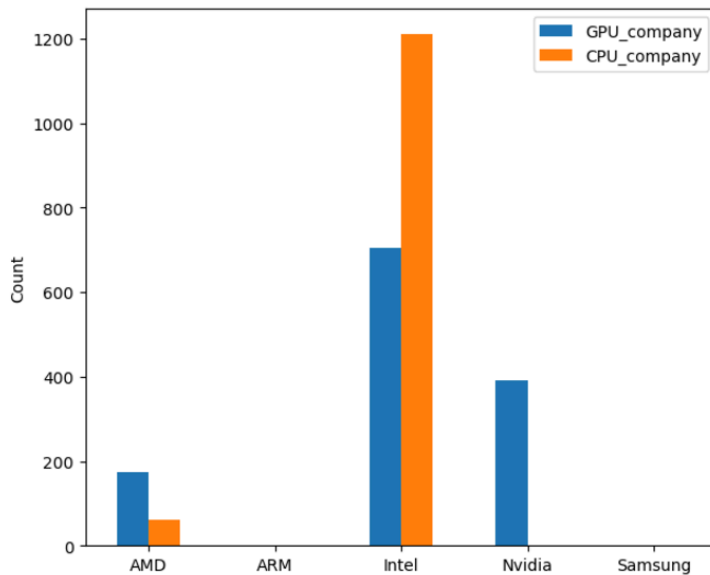


Figure 6.3.2: GPU Vs CPU Market Share



Additionally, Laptops with an Intel built CPU had a significantly higher average price compared to their next largest competitor AMD, with a median price of 1,010.26 vs 469 respectively (see figure 6.3.3 & 6.3.4). Similar to Windows OS, this highlights the popularity of Intel CPUs and should inform new product development.

Figure 6.3.3: Price Distribution Vs CPU Manufacturer

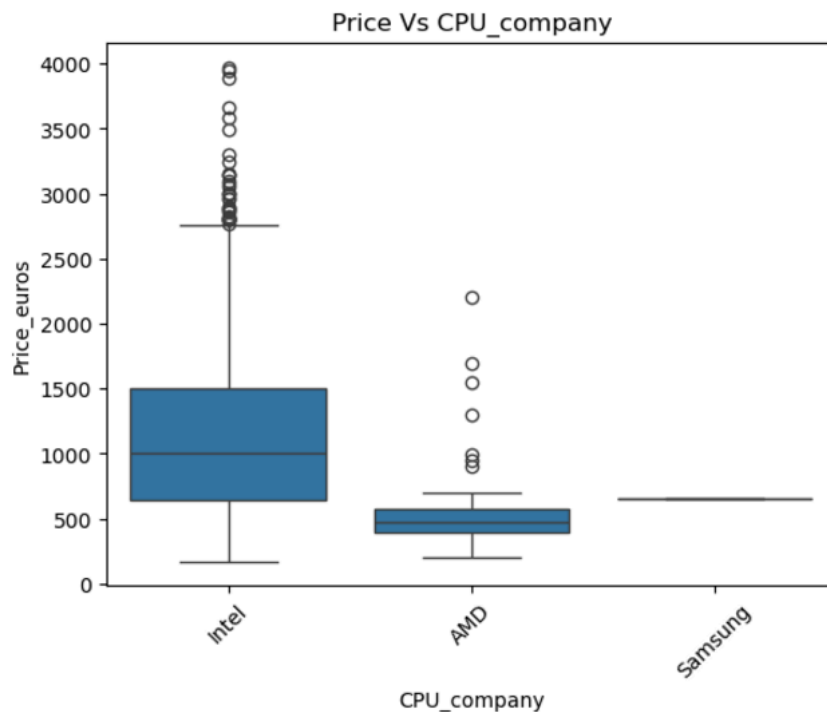


Figure 6.3.4: Summary Statistics of Fig. 6.3.3

	Mean	Std	Min	25%	50% (Median)	75%	Max
Intel	1150.314810	661.876719	174.0	649.00	1010.255	1499.0	3975.0
AMD	560.993833	350.347274	199.0	397.25	469.000	576.0	2199.0
Samsung	659.000000	NaN	659.0	659.00	659.000	659.0	659.0

4. Clustering of HDD, SSD, and Flash Storage

Figure 6.4.1 demonstrates that there are clusters forming when plotting primary storage against price and storage type. Flash storage presents a low-cost, low storage solution for economy class computers, SSD represents a high cost, low storage model, and HDD represents the low cost but high capacity model. This supports the performance narrative of Nelson (2024) that SSD provides improved performance. It also demonstrates the potential for a classification model for predicting if a PC is using HDD, SSD or Flash storage.

Figure 6.4.1: Swarmplot of Price Vs Storage Capacity

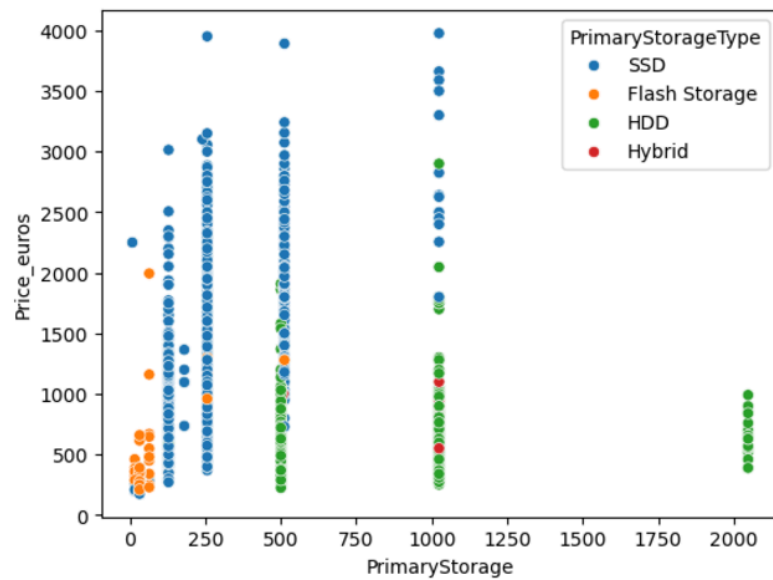


Figure 6.4.1 Continued: Swarmplot (With Lines Added to Classify the observation)

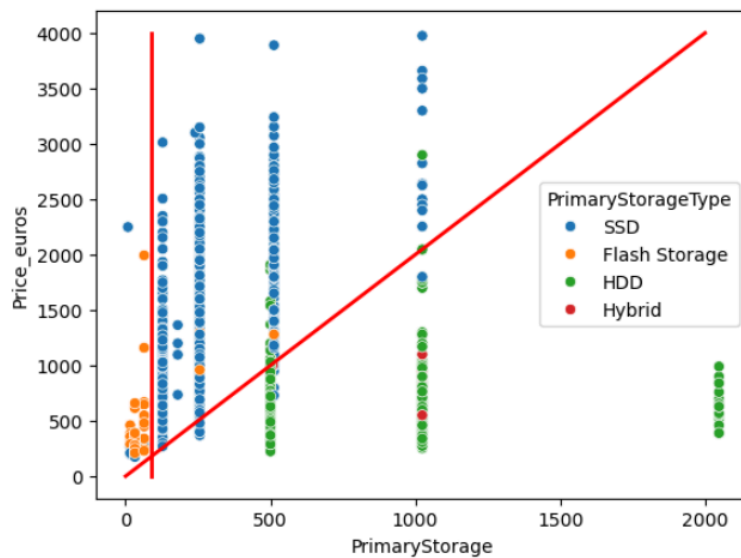


Figure 6.4.1 Continued: Confusion Matrix

PrimaryStorageType	prediction	
Flash Storage	Flash Storage	57
	SSD	14
HDD	Flash Storage	1
	HDD	90
SSD	SSD	268
	Flash Storage	11
	SSD	822

5. FullHD is More Popular Than Standard Definition

Despite its namesake, standard resolution is not the most common form of screen type, with over 65% of computers featuring Full HD and only around 29.03% featuring standard definition (see figure 6.5.1). This is despite the increased average price of computers featuring this setting as highlighted in figure 6.5.2 & 6.5.3, with median prices of €1101.50 compared to the market average of €985. This suggests that there is considerable demand for high-quality screens within the PC market, even outside of the premium segment of the market.

Figure 6.5.1: Screen Resolution Breakdown

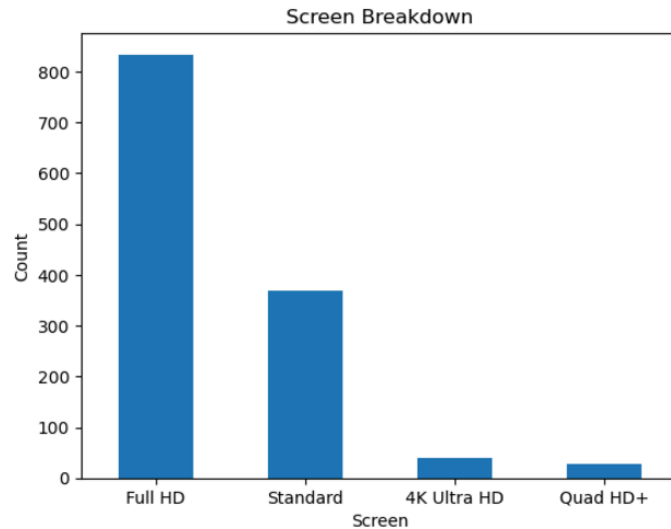


Figure 6.5.2: Box Plot of Price Against Screen Resolution

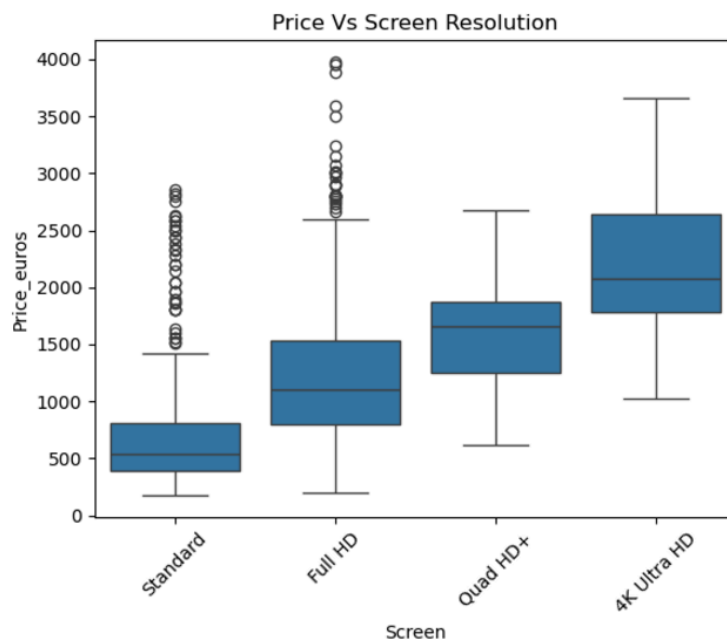


Figure 6.5.3: Box Plot Figures In Tabular Form

#Get summary for statistics for Prices grouped by Screen boxplot_statistics('Screen')							
	Mean	Std	Min	25%	50% (Median)	75%	Max
Standard	730.325122	557.902938	174.0	387.00	540.000	805.99	2858.0
Full HD	1227.418441	602.012934	196.0	799.00	1101.495	1535.00	3975.0
Quad HD+	1616.955000	448.622413	615.0	1250.75	1655.320	1876.50	2680.0
4K Ultra HD	2194.196500	647.639560	1029.0	1786.50	2074.000	2641.50	3659.4

6. Workstation, Ultrabook and Gaming PCs are, on average, more expensive than Other Computers.

As demonstrated from Figure 6.6.1, PCs of type Gaming, Workstation and Ultrabook have median prices of €1,486.77, €2028.95, and €1499 respectively, higher than the median price of the entire dataset of €985. This is also the case for the 1st and 3rd quartiles, as illustrated in 6.6.2, and 6.6.3. This may provide a method of segmenting the PC market, with the segments highlighted above as the potentially more lucrative markets.

Figure 6.6.1: Box Plot Showing Price Against Laptop Type

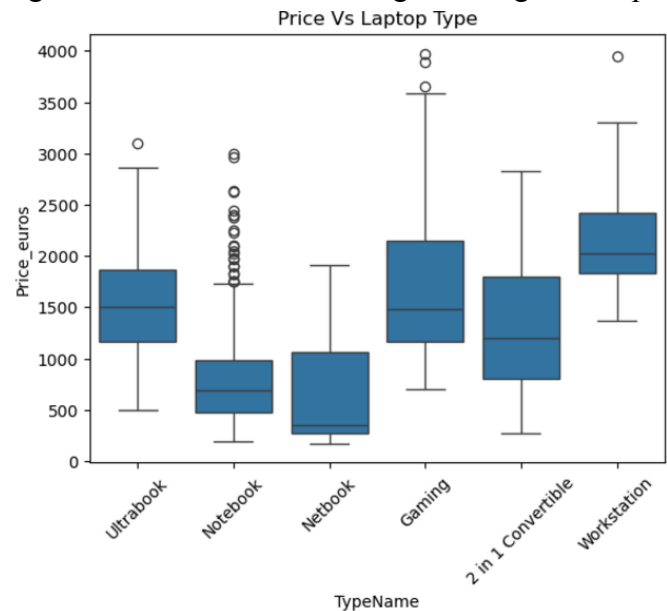


Figure 6.6.2: Summary Statistics of Price

#Get summary for statistics for Prices get_continuous_details(dfNumeric[["Price_euros"]])								
	count	mean	std	min	25%	50%	75%	max Unique
Price_euros	1271.0	1122.108222	662.225815	174.0	606.0	985.0	1491.4	3975.0 787

Figure 6.6.3: Summary Statistics of Price Grouped by Typename

```
#Get summary for statistics for Prices grouped by Typename
boxplot_statistics('TypeName')
```

	Mean	Std	Min	25%	50% (Median)	75%	Max
Ultrabook	1556.676031	484.737332	499.0	1159.7750	1499.000	1869.000	3100.0
Notebook	782.922890	419.448877	196.0	478.9175	694.495	979.750	2999.0
Netbook	673.382609	592.093391	174.0	269.0000	355.000	1066.500	1908.0
Gaming	1691.305567	709.310241	699.0	1168.5000	1486.770	2151.685	3975.0
2 in 1 Convertible	1289.714530	617.436633	275.0	798.0100	1199.000	1799.000	2824.0
Workstation	2205.050357	596.748493	1369.0	1835.7500	2028.950	2425.500	3949.4

7. All 4 Datasets Had a Positive Skew and Multiple Outliers

For all datasets observed as part of this project, each had a price feature that had a positive skew, meaning that the distribution was distorted by a number of outliers that have values much greater than the mean. This is reflected in the fact that the median price is less than the mean price of the product (see figure 6.7.2).

Figure 6.7.1: Price Distribution Across all Four Datasets

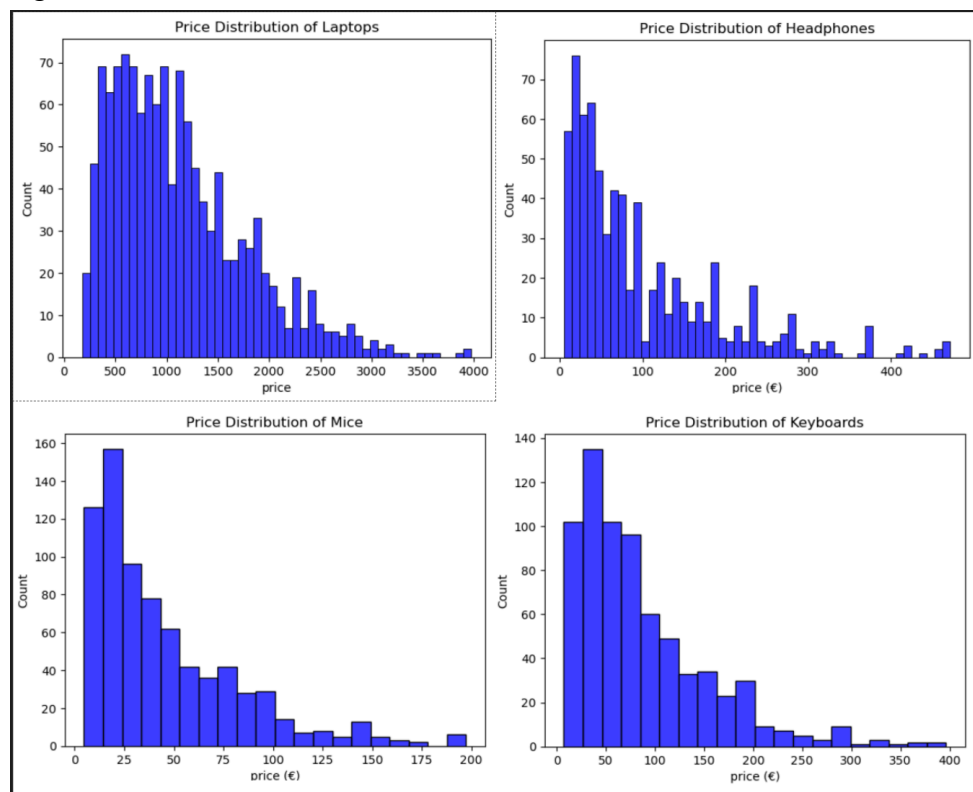


Figure 6.7.2: Median, Mean, Standard Deviation and Max Prices of All Datasets .

Details	Median	Mean	Standard Dev.	Max
Laptop Prices	€985.00	€1,122.11	€662.23	€3975.00
Headphones	€66.02	€98.36	€93.44	€471.85

Mice	€33.78	€45.86	€36.90	€197.25
Keyboards	€66.05	€86.94	€54.68	€396.39

Despite the removal of outlier values, the data is still skewed by a small number of products with disproportionately higher prices, often over 3-4 standard deviations above the mean. This might be an indication of price gouging or “scalping”, a potential business risk where customers are purchasing PC goods and reselling them at an inflated margin, resulting in the outlier. An example has been included in Appendix 4.

8. The Vast Majority of Gaming Keyboards Had Built-In Lighting

Following Figure 6.8.1, Figure 6.8.2 was created to assess if a keyboard having lights would impact the price, and if there was a correlation between type and backlight. The graph shows that the overwhelming majority of gaming keyboards have some lighting effect, which illustrates the designed functions of a keyboard within the Gaming PC market.

Figure 6.8.1: Backlight Analysis

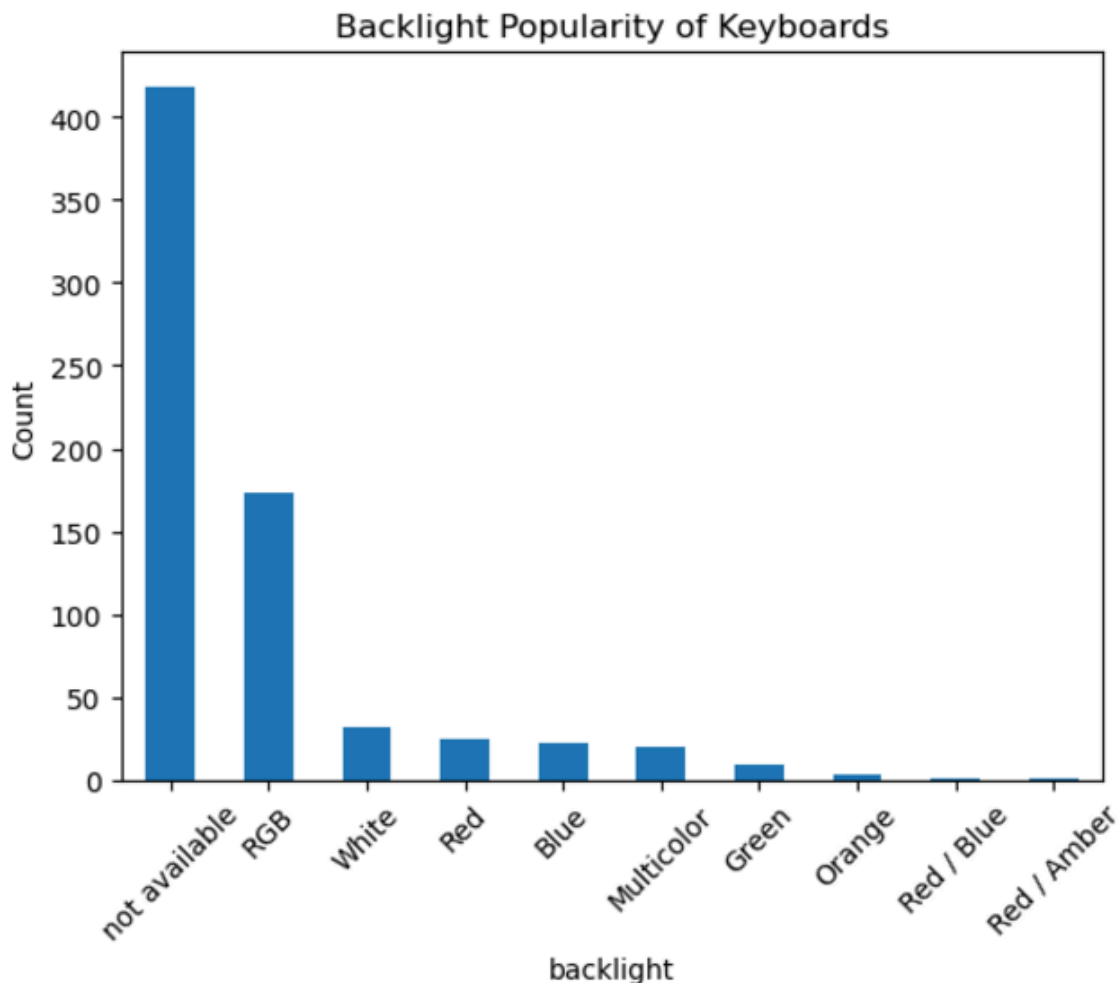
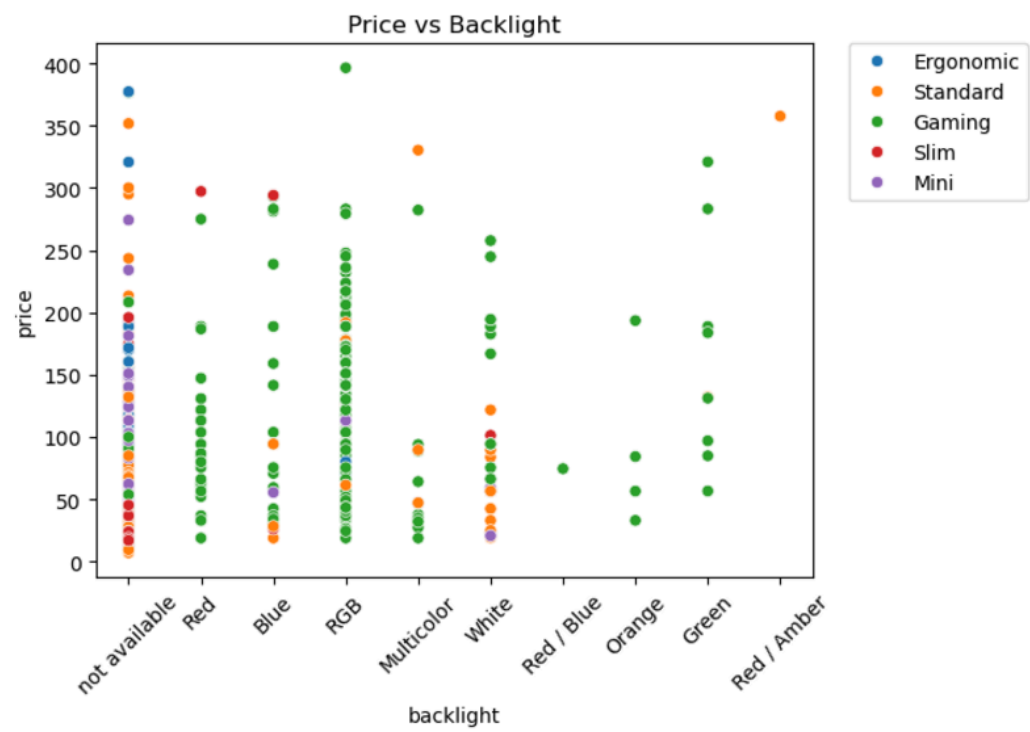


Figure 6.8.2: Swarmplot of Price Against Keyboard Style and Lighting



7. Machine Learning

The primary goal of the model is to predict the price of a PC based on the various parts. The optimum means of achieving this is through a regression algorithm, as it is designed to predict continuous numeric variables such as pricing. Classification would be less relevant, as it is ideal when the target feature is categorical (either one of two states) but could be applied to refine the classification attempt in Figure 6.4.1.

The regression model will be trained using features identified in section 6 to evaluate if features such as RAM will have an impact on the price of PCs. This will enable PC retailers to anticipate the price a consumer is willing to pay for their product, and assist in revenue forecasting, and error metrics such as RMSE or MSE can be used to measure the expected error in assessing price. If the classification route is explored, a confusion matrix could be generated to assess the accuracy similar to figure 6.4.1.

A similar model could be implemented for the headphones, mice, and keyboards datasets, or incorporated into the primary model in order to determine changes in price if these features are to be packaged with the PC as complementary goods.

8. Conclusion

The analysis and cleaning of the dataset has identified features relevant to training models, as well as potential trends and business risks associated with entering the PC market.

Bibliography:

Corsair (2024) *The Anatomy of a PC - What's inside, and what does each part do?* Available at: <https://www.corsair.com/us/en/explorer/diy-builder/blogs/the-anatomy-of-a-pc/> (Accessed 21 October 2024).

Edmonds, R. (2023) "What is TDP how does it affect your PC's performance?", *Windows Central*, 17 November. Available at: <https://www.windowscentral.com/what-tdp-and-why-should-you-care-about-it> (Accessed 14 November 2024).

Financial Times (2024) *Euro/US Dollar FX Spot Rate*. Available at: <https://markets.ft.com/data/currencies/tearsheet/summary?s=EURUSD> (accessed 19 November 2024).

Griffith, E (2021) "The Desperate Hunt for the A.I. Boom's Most Indispensable Prize", *NYT*, 16 August. Available at: <https://www.nytimes.com/2023/08/16/technology/ai-gpu-chips-shortage.html> (Accessed 21 October 2024).

Hawthorne, M. (2024) *Base Clock and Boost Clock Explained*. Available at: <https://www.technipages.com/base-clock-and-boost-clock-explained/#:~:text=Base%20Clock%20The%20clock%20speed%20is%20a%20measure,the%20processor%20can%20run%20at%20with%20reasonable%20cooling.> (Accessed 14 November 2024).

IBM (2021) *CRISP-DM in IBM SPSS Modeler*. Available at: <https://www.ibm.com/docs/sr/spss-modeler/saas?topic=overview-crisp-dm-in-spss-modeler> (Accessed 21 October 2024).

Molloy, D (2021) "The great graphics card shortage of 2020 (and 2021)", *BBC*, 24 January 2021. Available at: <https://www.bbc.com/news/technology-55755820> (Accessed 21 October 2024).

Nelson, K. (2024) *SSD vs HDD: Choosing the Best Storage for Your Needs*. Available at: <https://www.hp.com/us-en/shop/tech-takes/ssd-vs-hdd> (Accessed 19 November 2024).

Wagih (2024) *Laptop Prices*. Available at: <https://www.kaggle.com/datasets/owm4096/laptop-prices?resource=download> (Accessed 7 October 2024).

Wu, K (2021) *py-moneyed: Usage*. Available at:
<https://py-moneyed.readthedocs.io/en/latest/usage.html> (accessed 18th November 2024).

Short, C (2010) "How User-Friendly Operating Systems and Interfaces Make Technology Harder for for Everyone", *University of Lynchburg*, Spring, pp 1-25 (accessed 8 December 2024)

Simon (2024) *nynhex/PCPartPicker-API*. Available at:
<https://github.com/nynhex/PCPartPicker-API> (Accessed 21 October 2024).

Appendix 1: Graphical Analysis

Univariate

Laptop Prices

Figure 5.1.9.1: Laptop Prices Feature Breakdown:

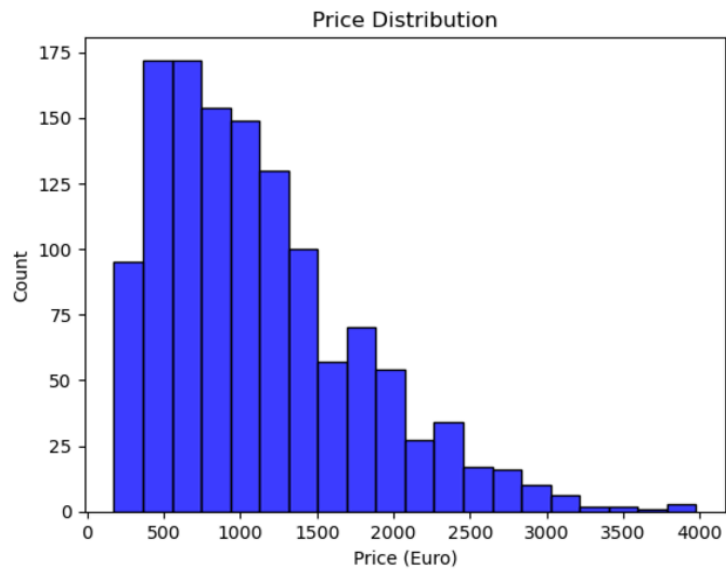


Figure 5.1.9.2: Laptop Prices Feature Breakdown (Boxplot version):

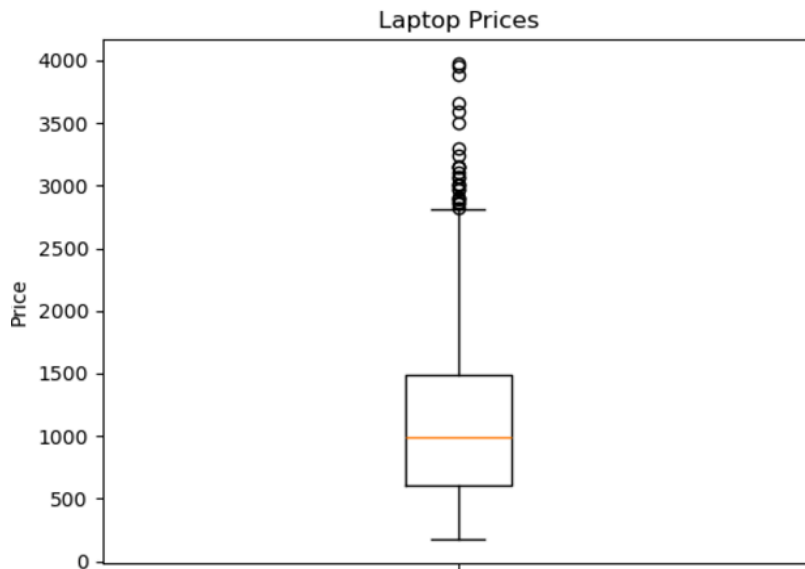


Figure 5.1.10: SSD Storage Breakdown

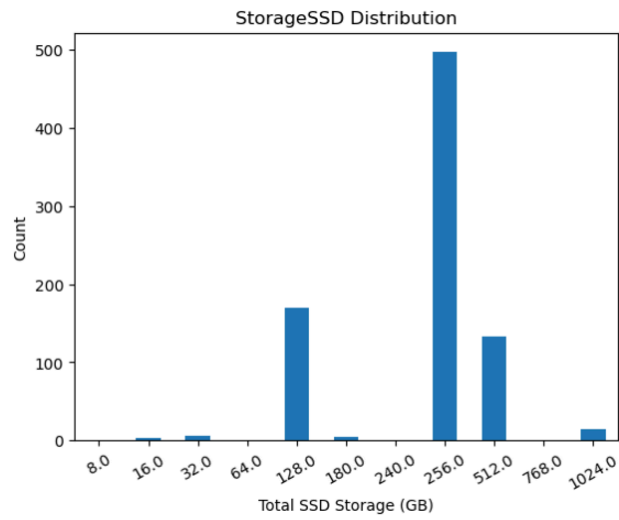


Figure 5.1.11: Flash Storage Breakdown

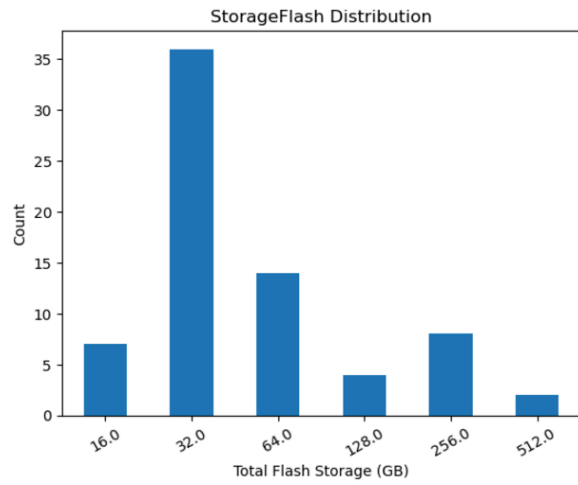


Figure 5.1.12: HDD Storage Breakdown

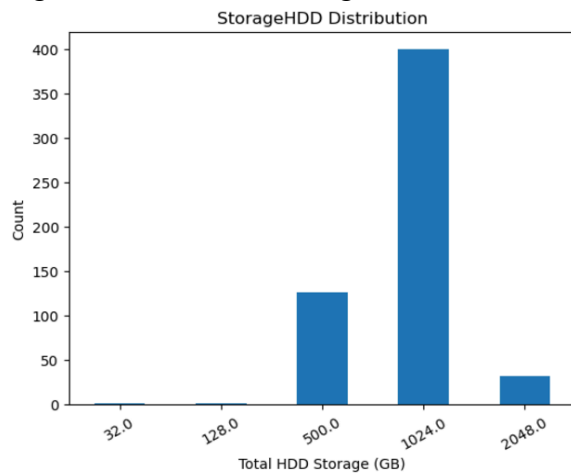


Figure 5.1.13: Hybrid Storage Breakdown

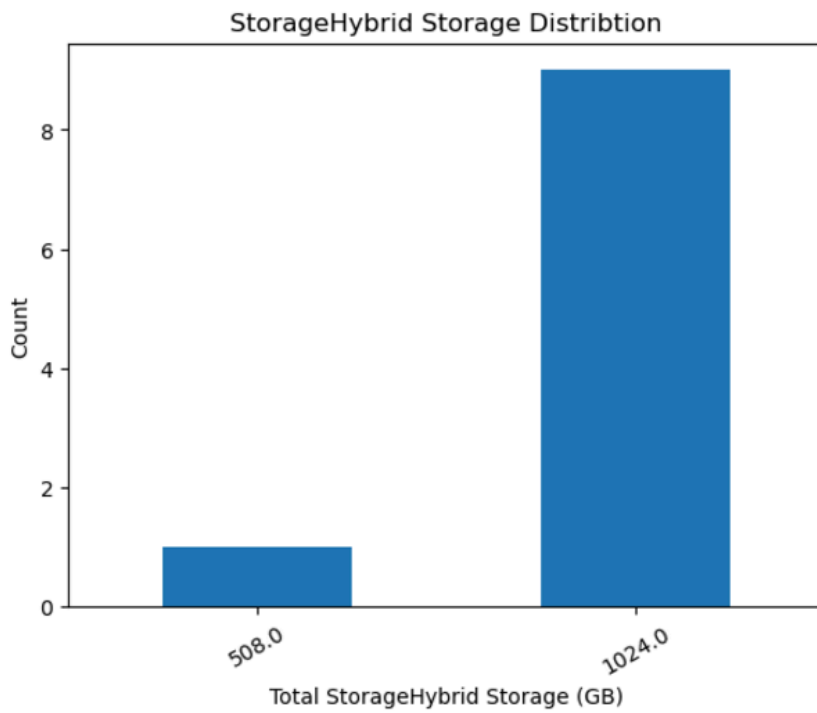


Figure 5.1.14: Total Storage Breakdown

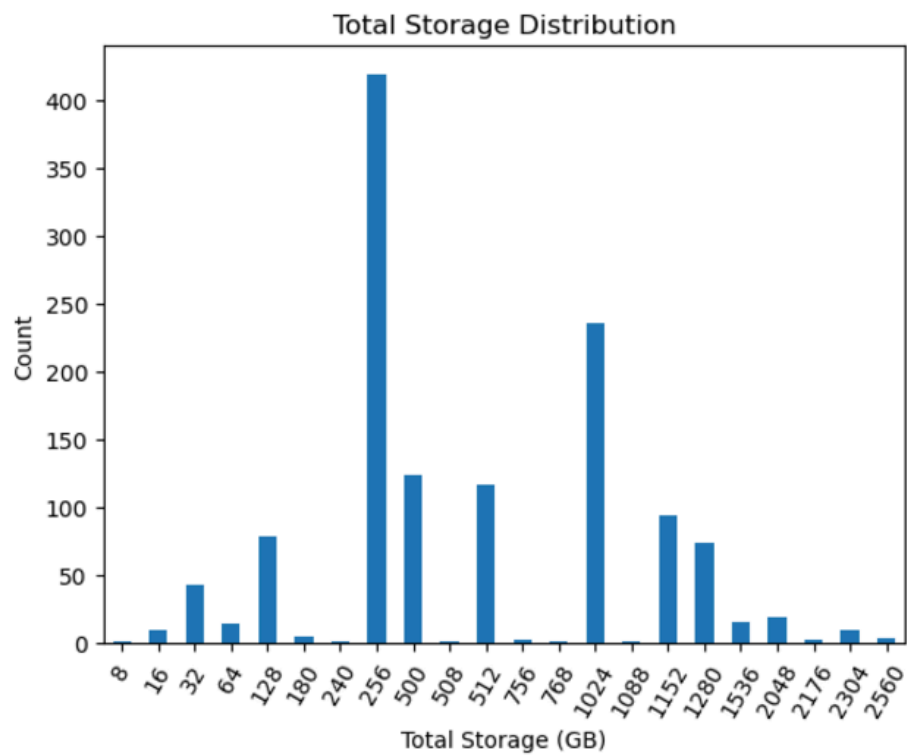


Figure 5.1.15: Manufacturer Breakdown

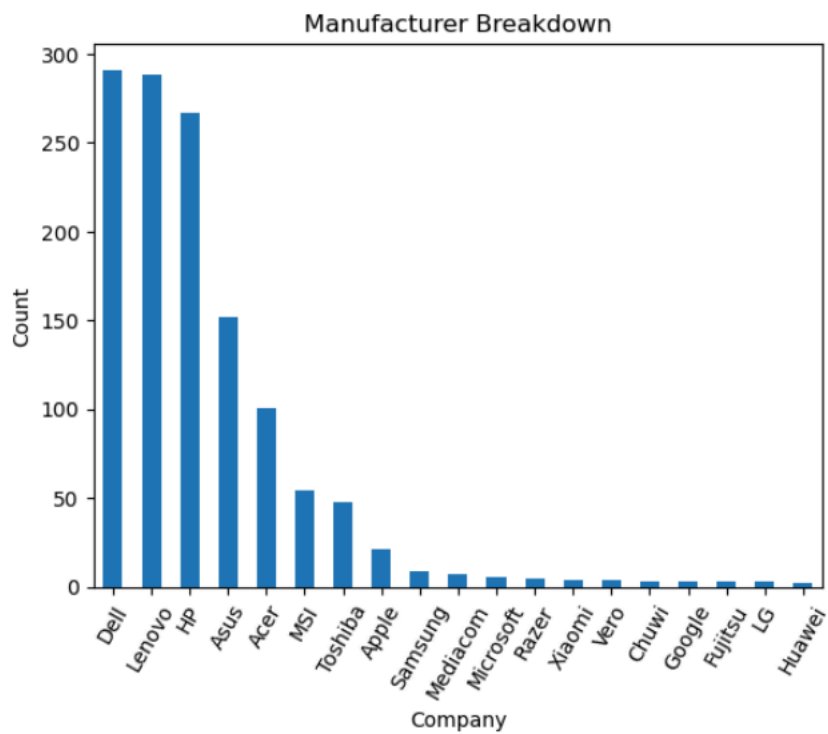


Figure 5.1.16: Laptop Category Breakdown

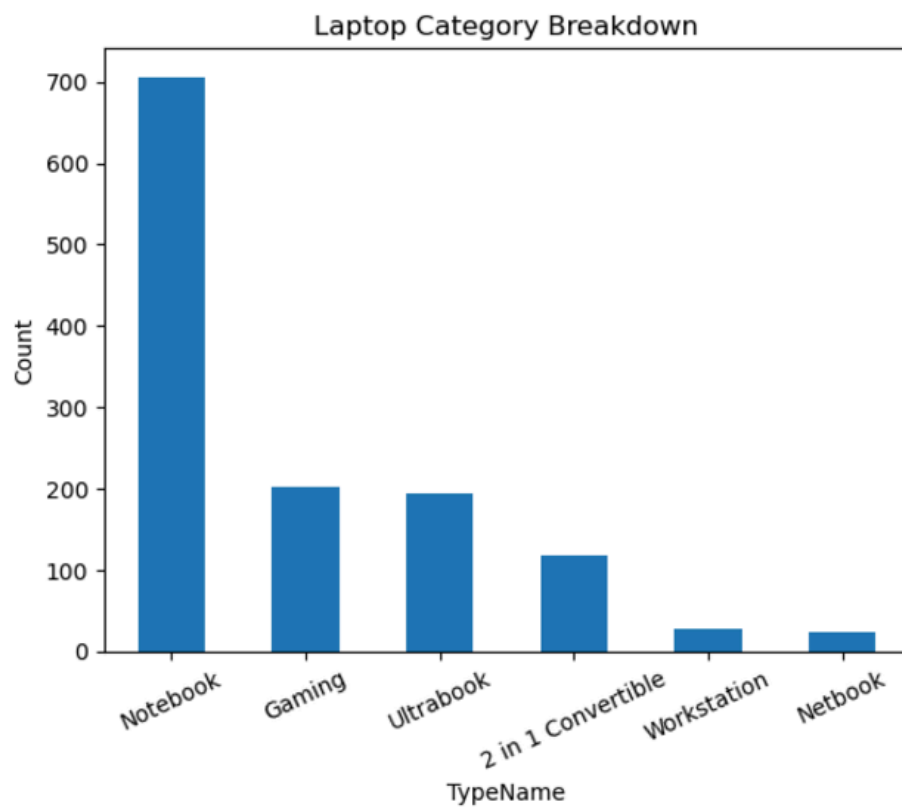


Figure 5.1.17: Operating System Breakdown

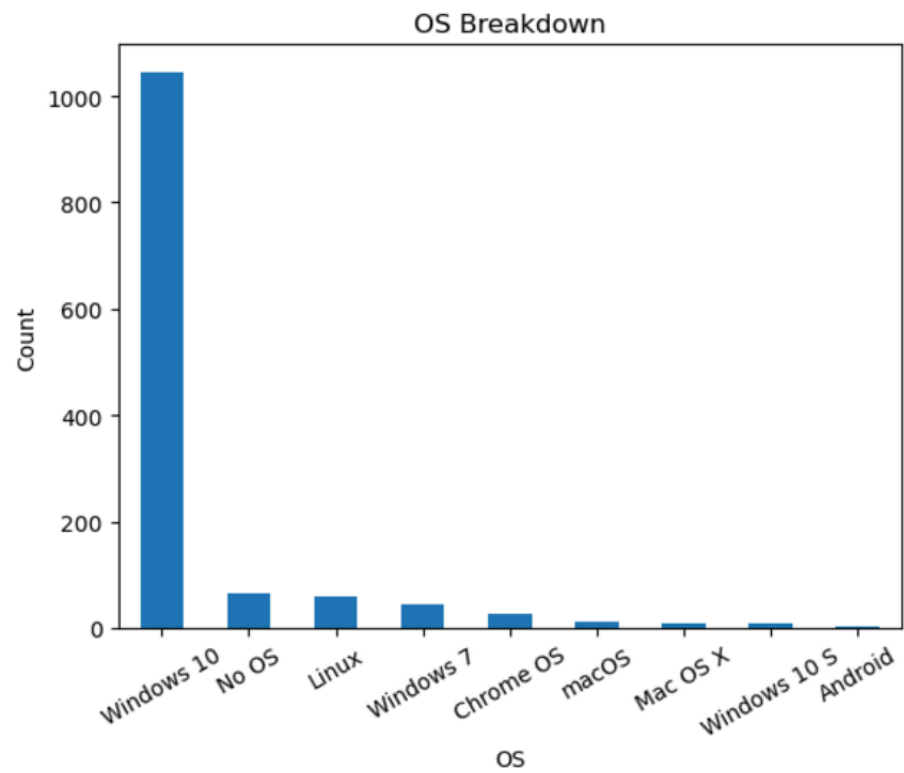


Figure 5.1.18: Screen Resolution Breakdown

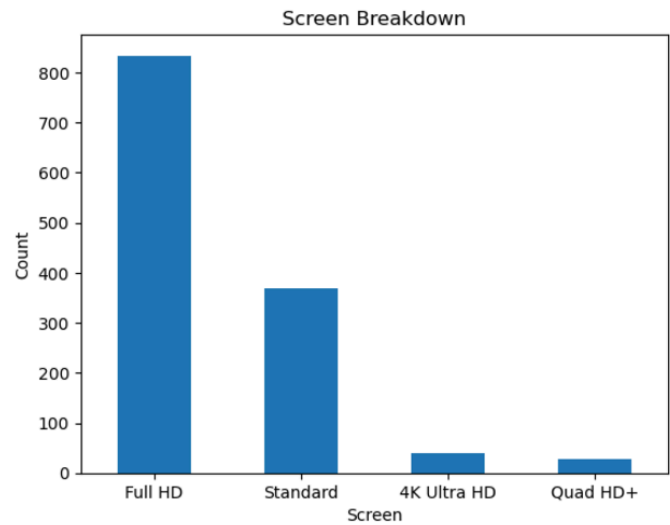


Figure 5.1.19: Touch Screen Computers

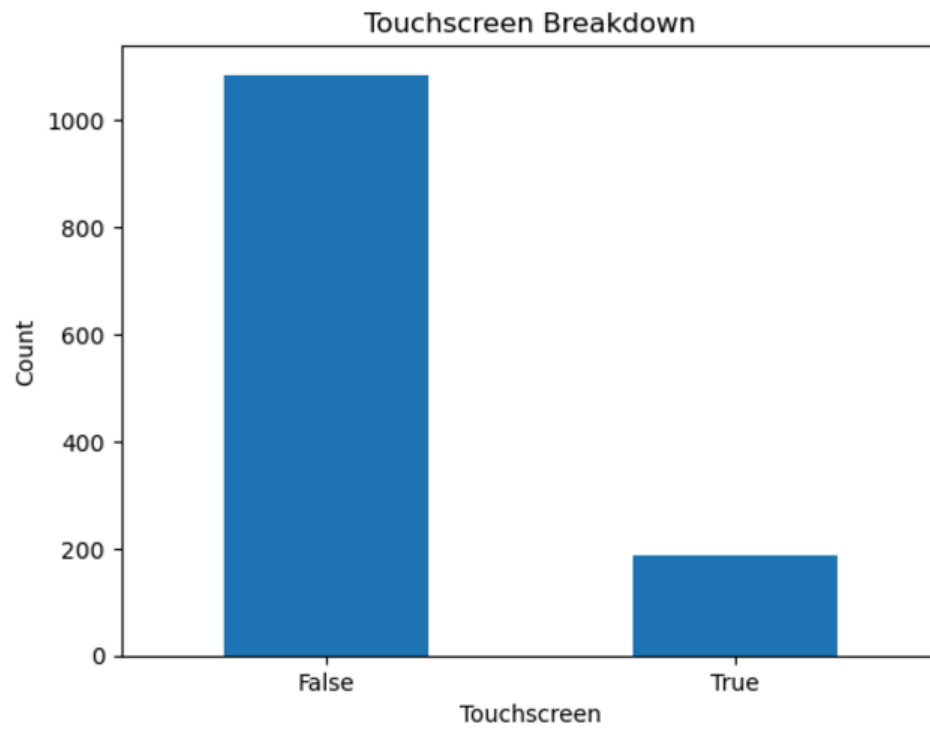


Figure 5.1.20: IPSpanel

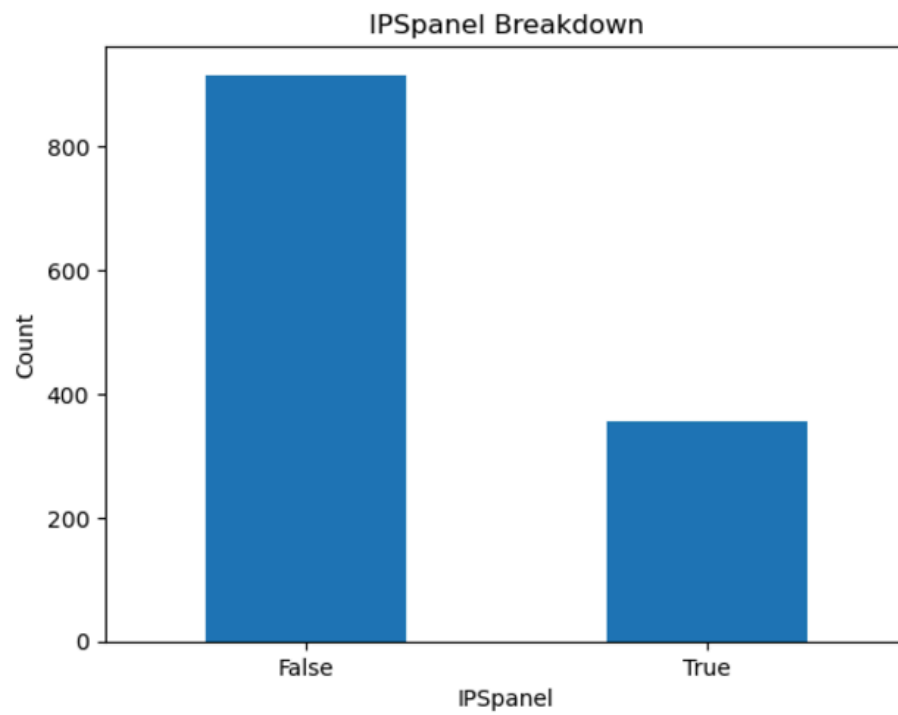


Figure 5.1.21: Retina Display

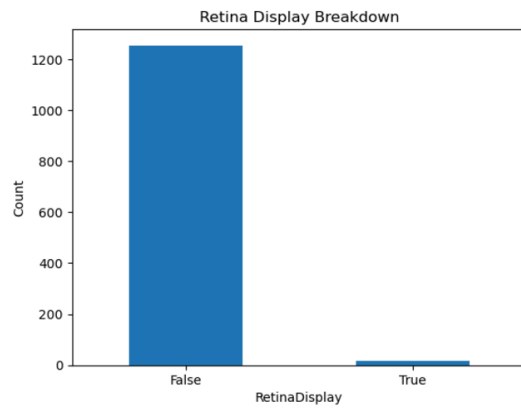


Figure 5.1.22.1: CPU Manufacturer Analysis

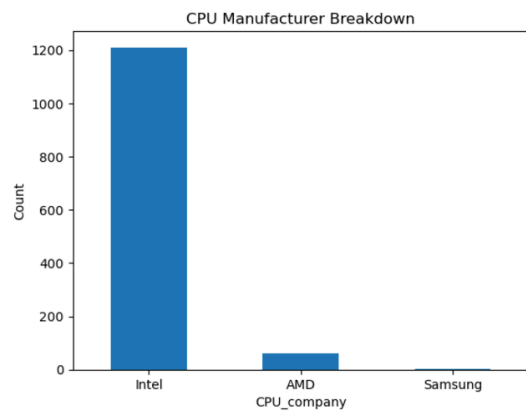


Figure 5.1.22.2: CPU Manufacturer Analysis (Pie Chart Format, Made with Matplotlib)

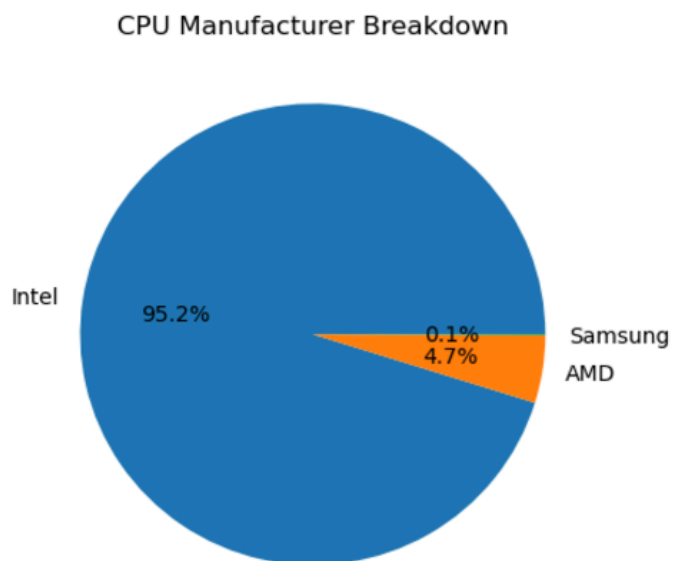


Figure 5.1.23: GPU Manufacturer

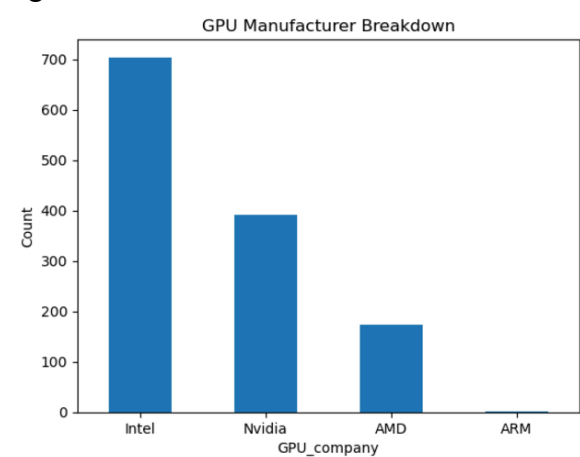


Figure 5.1.24: Primary Storage Breakdown

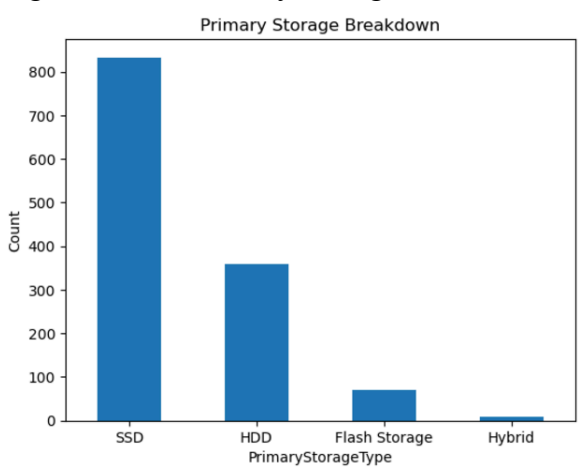


Figure 5.1.25: 10 Most Popular GPU Models

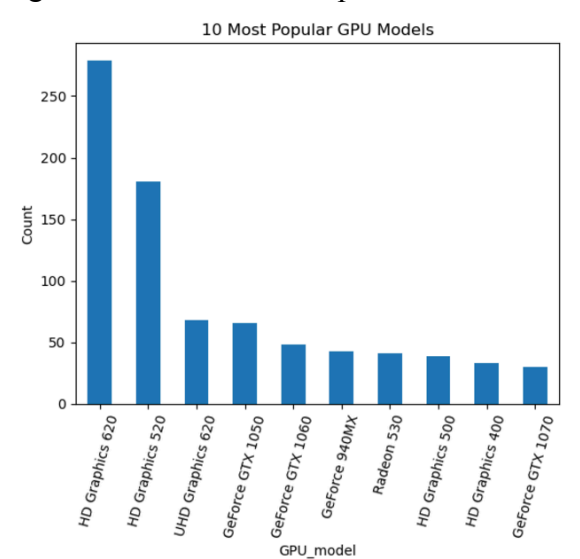
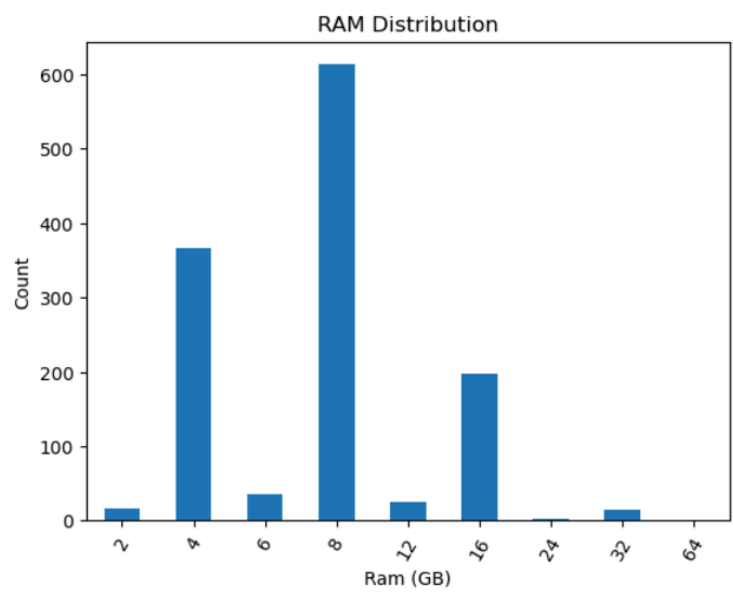


Figure 5.1.26: Ram Distribution



Headphones

Figure 5.1.27: Most Popular headphone Models

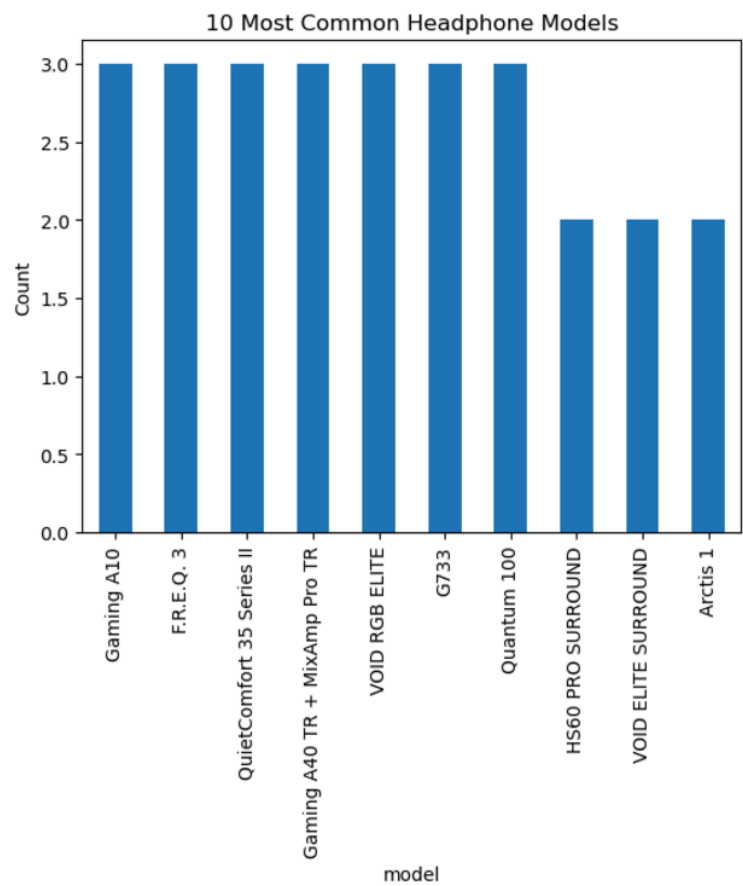


Figure 5.1.28: Form Factor Distribution

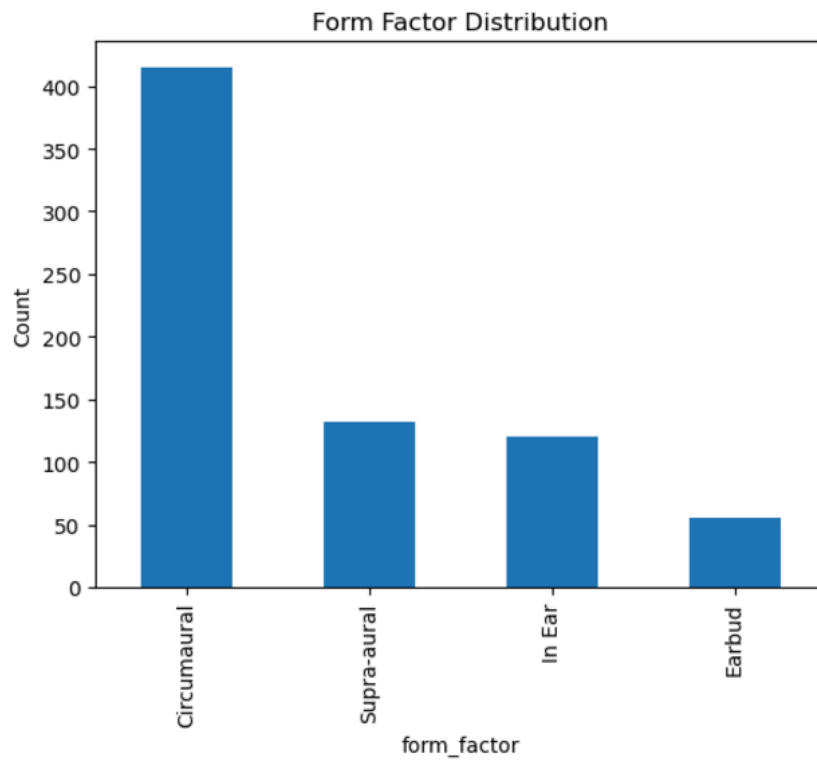


Figure 5.1.29: Commonality of Microphones on Headsets

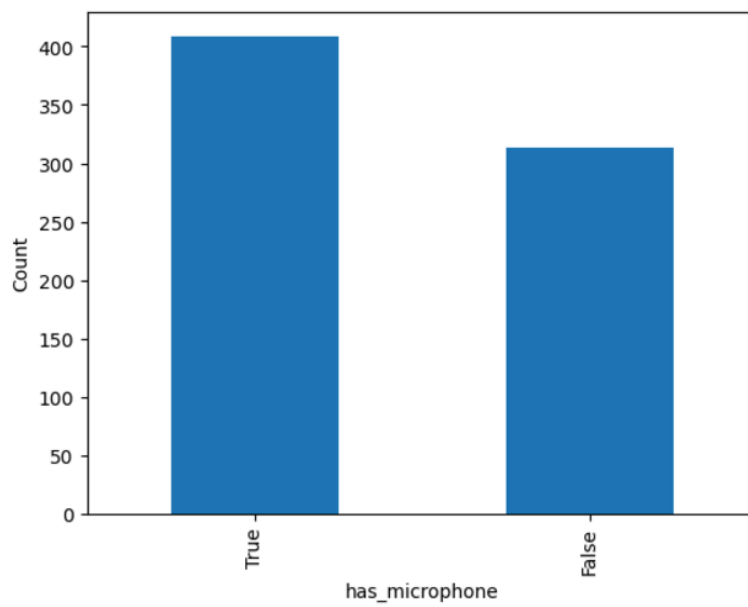


Figure 5.1.30: Popularity of Colour Among Headphone Models

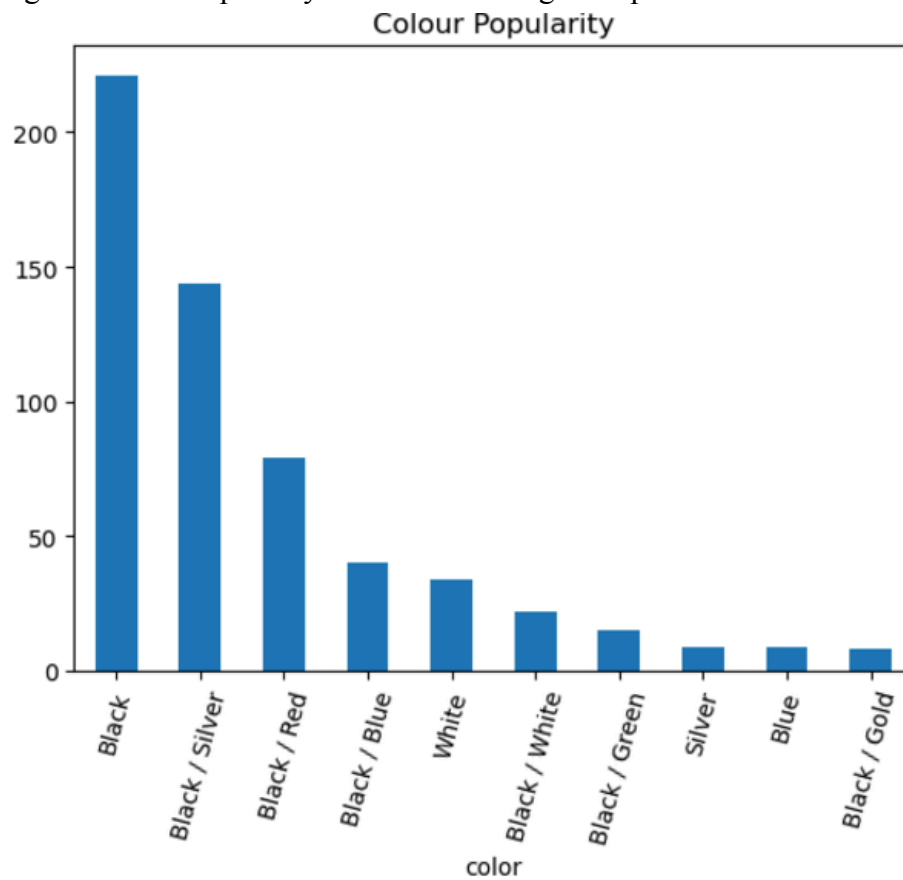
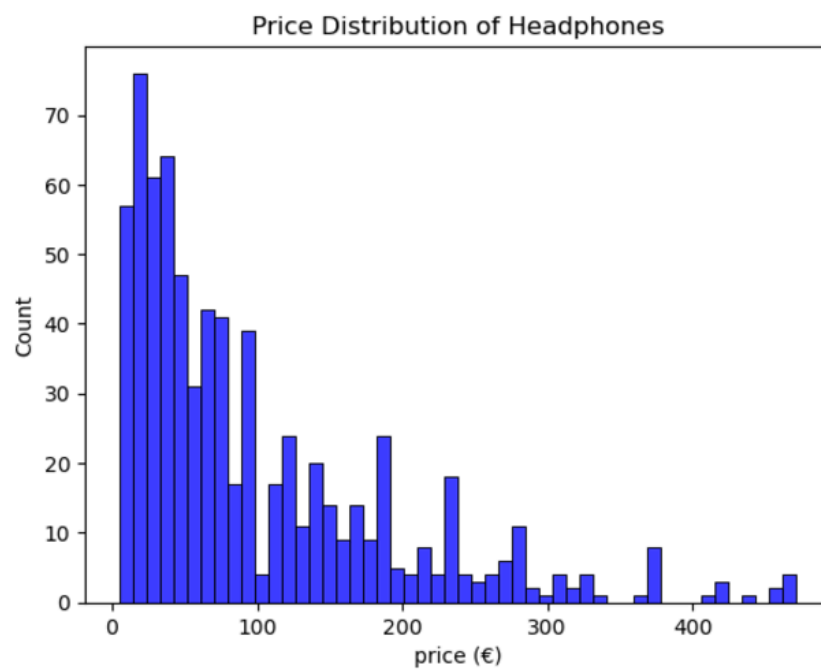


Figure 5.1.31: Price Distribution of Headphones



Mice

Figure 5.1.32: Tracking Feature Breakdown

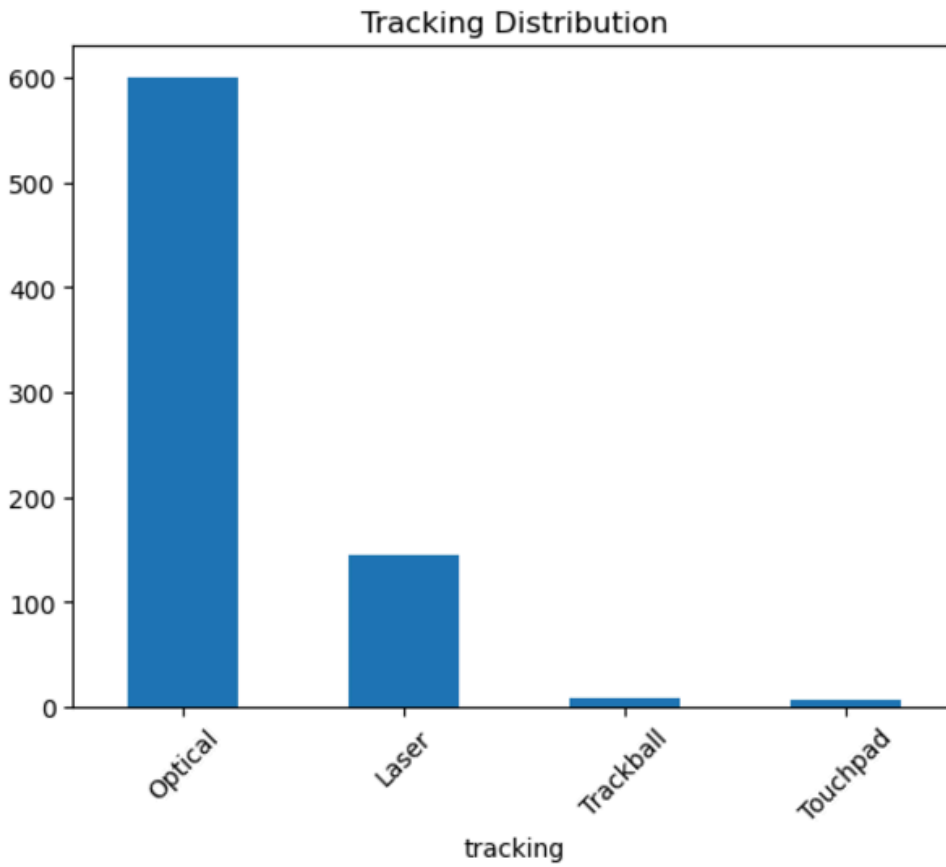


Figure 5.1.33: Connection Feature Breakdown

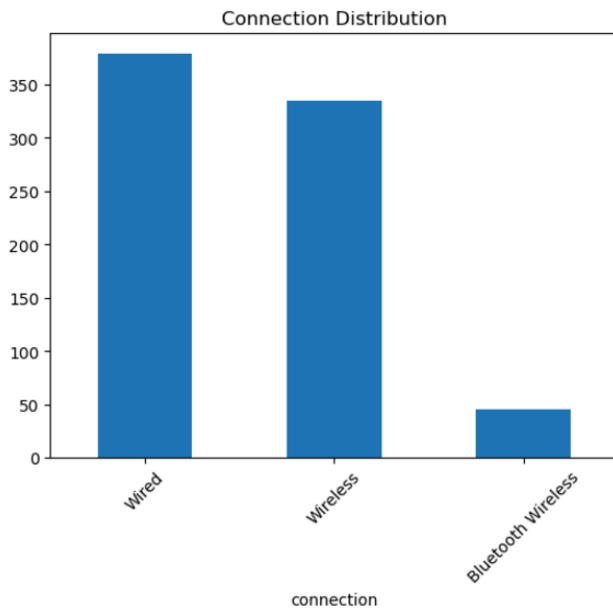


Figure 5.1.34: Hand Orientation

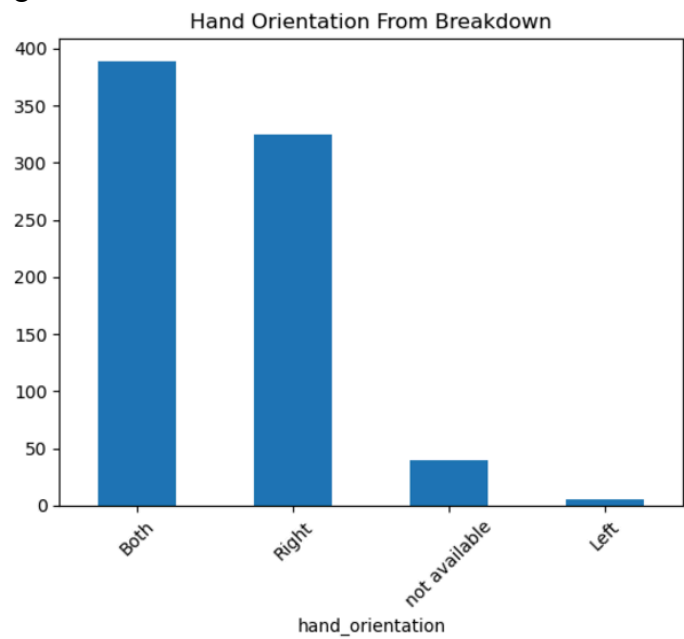


Figure 5.1.35: Colour Popularity for Mice

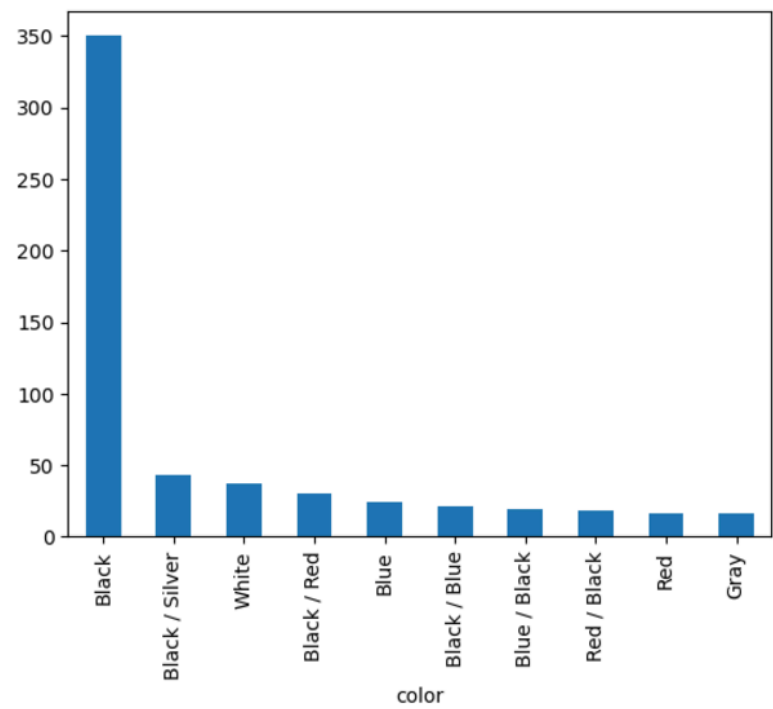
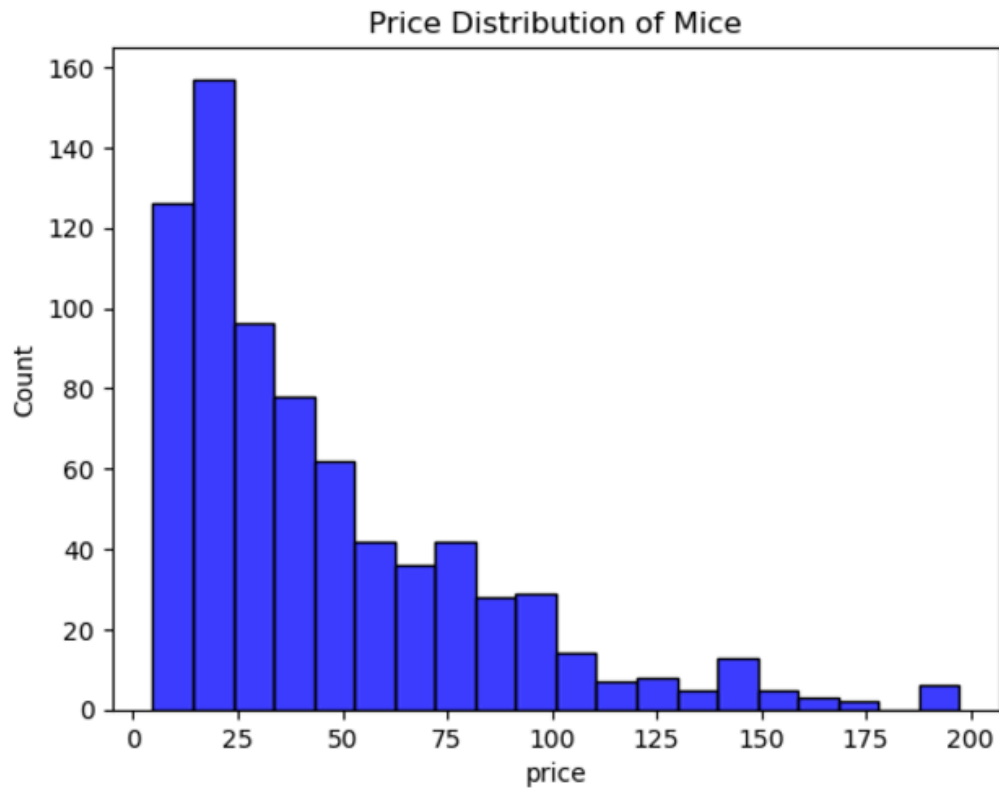


Figure 5.1.36: Price Distribution of Mice



Keyboards

Figure 5.1.37: Price Distribution of Keyboards

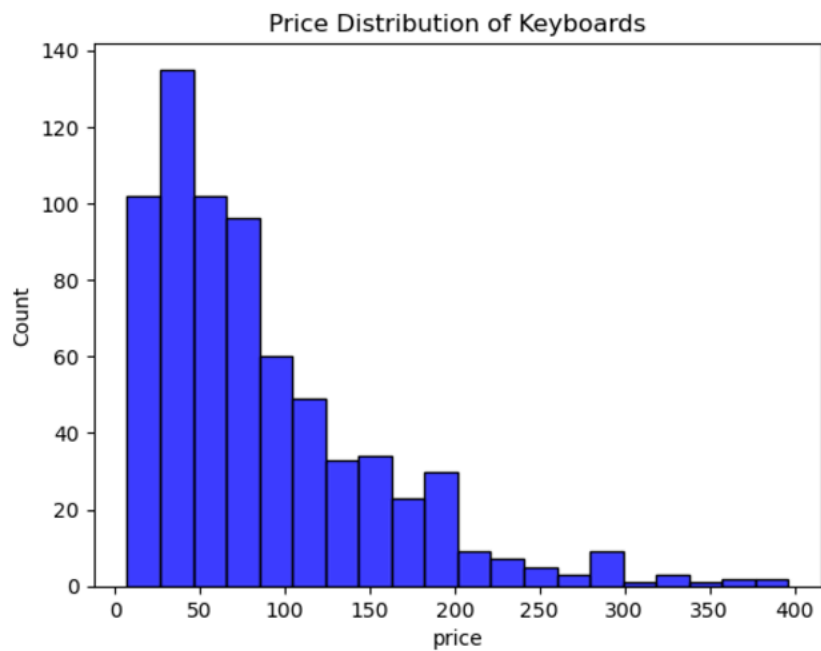


Figure 5.1.38: Keyboard Model Popularity:

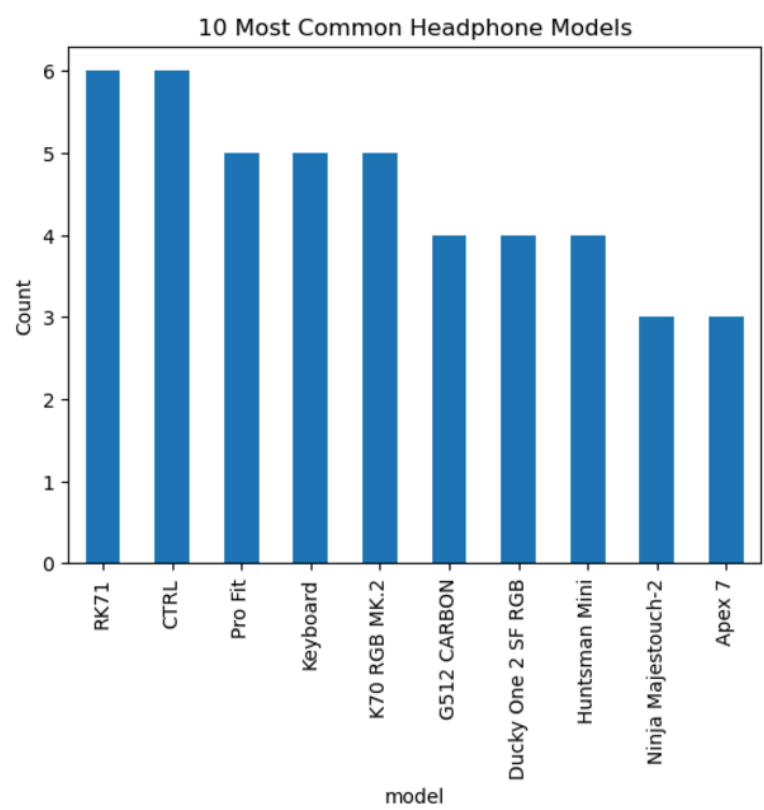


Figure 5.1.39: Style Breakdown

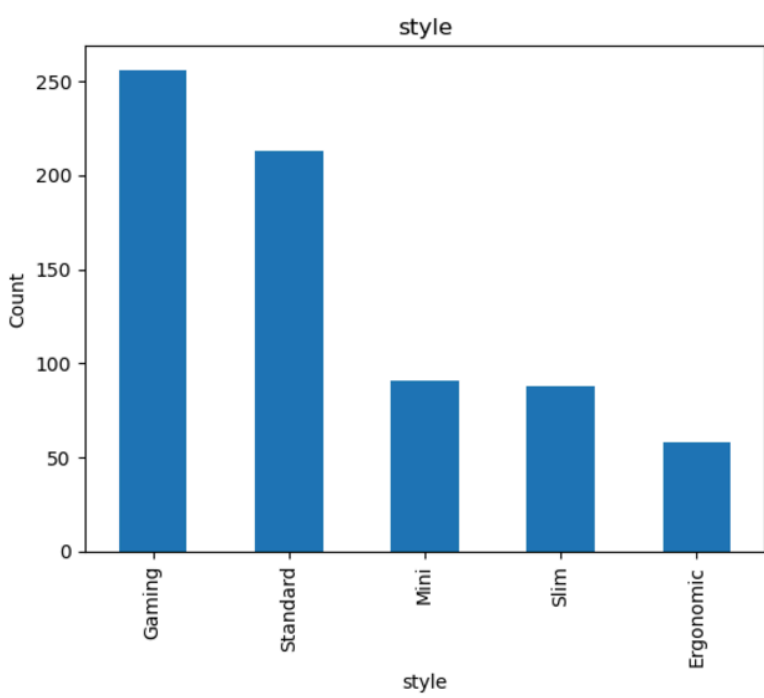


Figure 5.1.40: Style Breakdown

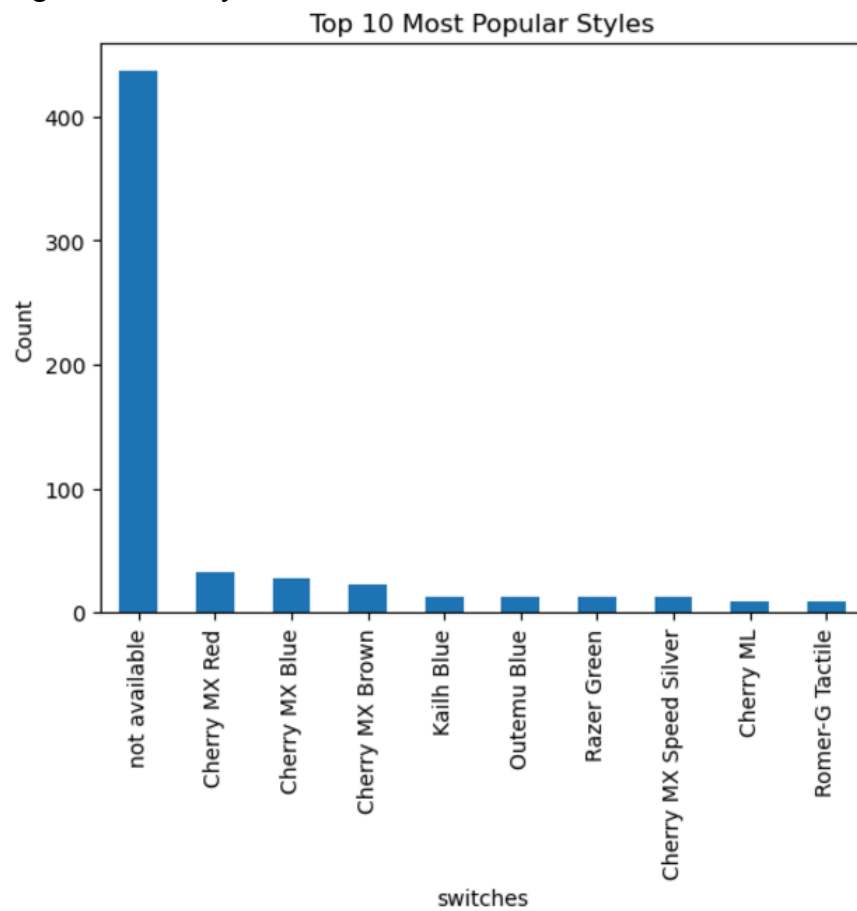
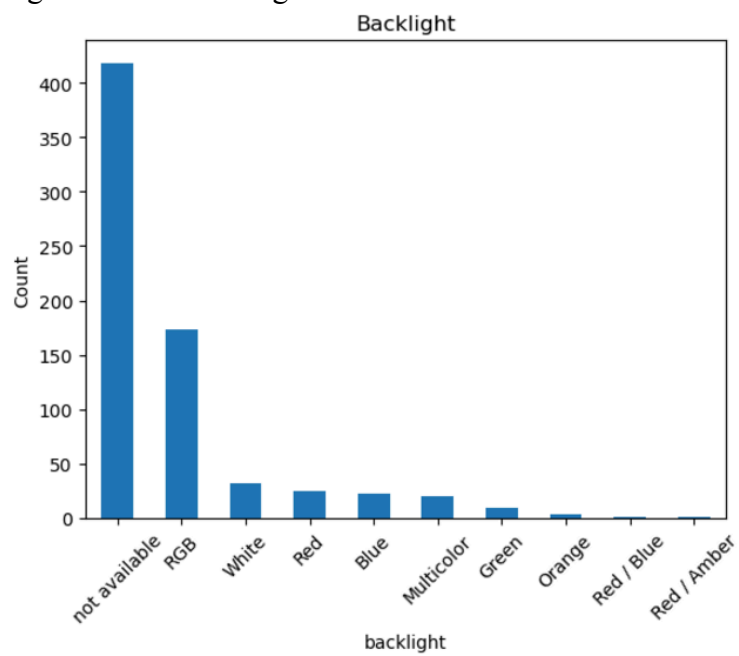


Figure 5.1.41: Backlight Breakdown



Bivariate

Laptop Prices

Figure 5.2.2: Heatmap of Correlation Coefficient of Numeric Features

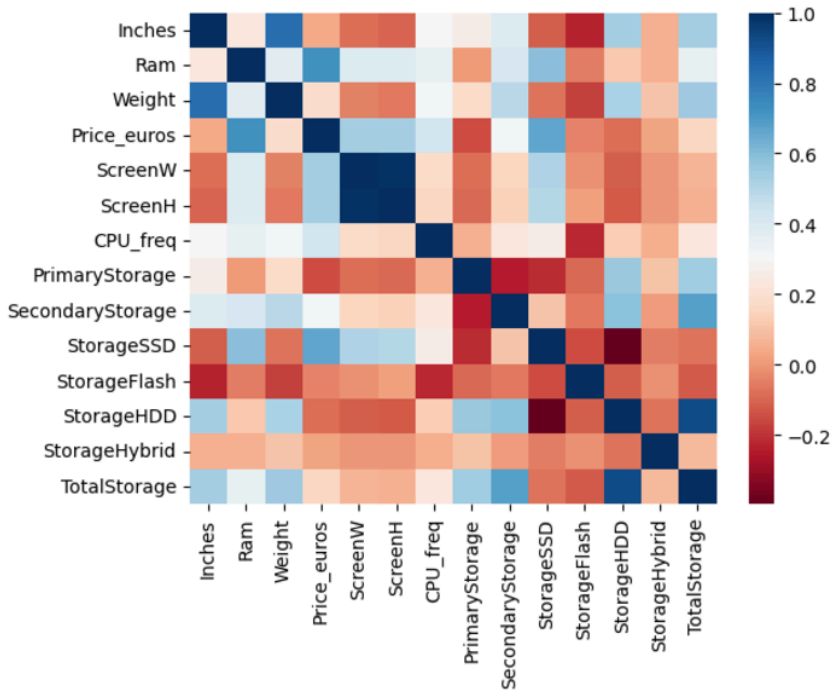


Figure 5.2.3: Box Plot of Price against Operating System

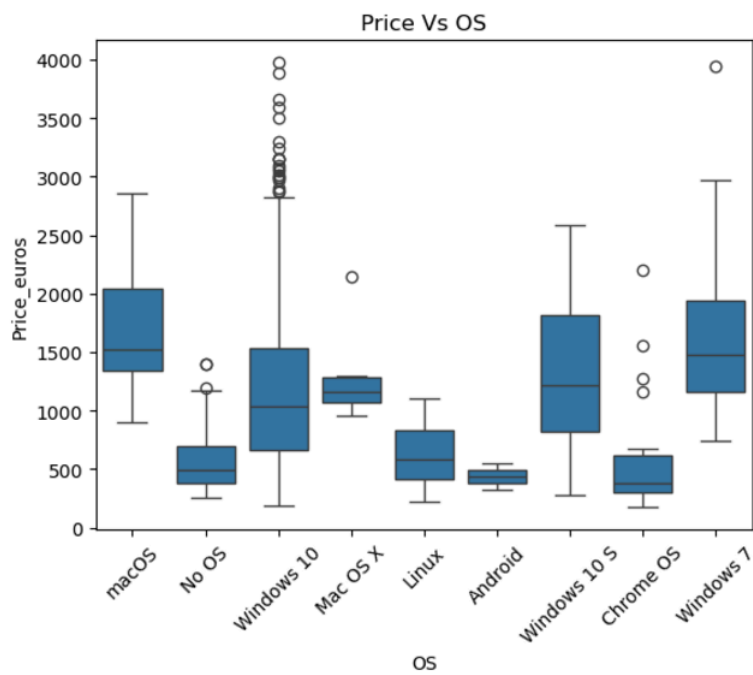


Figure 5.2.4: Swarm Plot of Price Vs Primary Storage Capacity (and type)

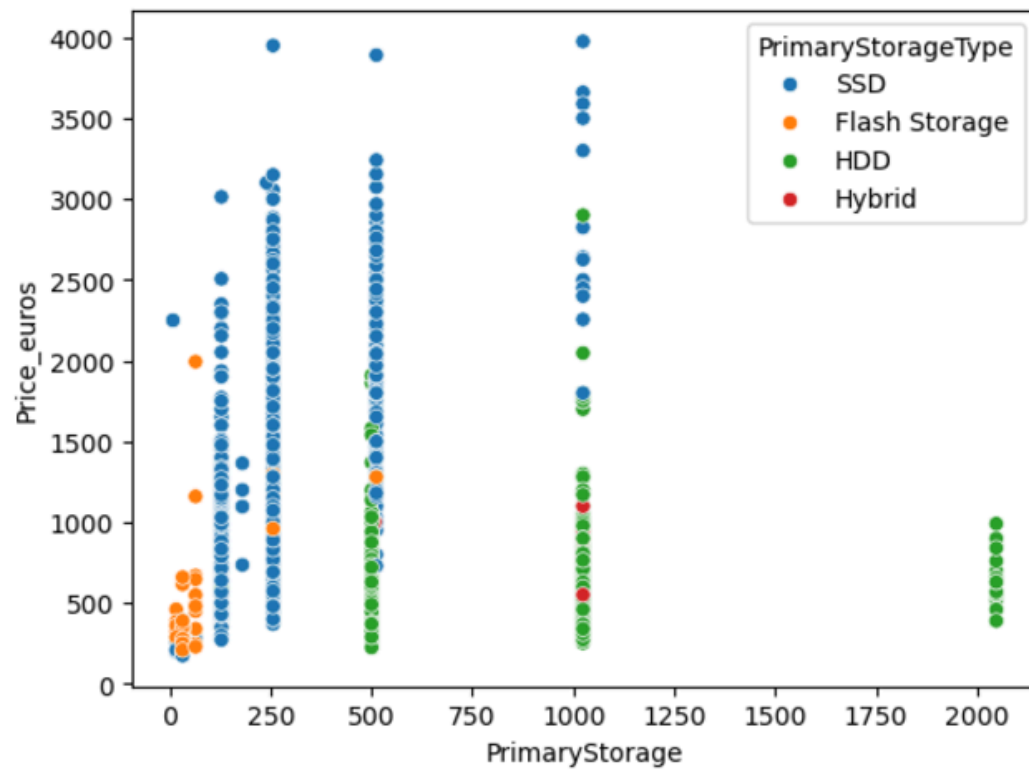


Figure 5.2.5: Strip Plot of Primary Storage Type Vs Primary Storage

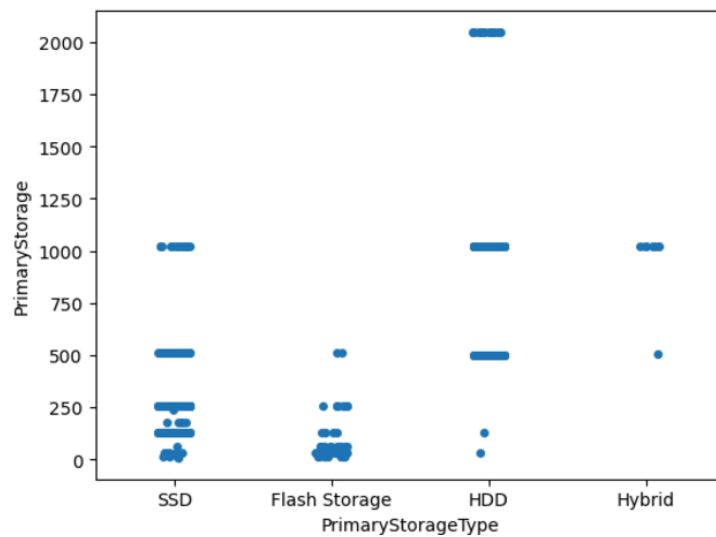


Figure 5.2.6: Price Distributions vs Laptop Type

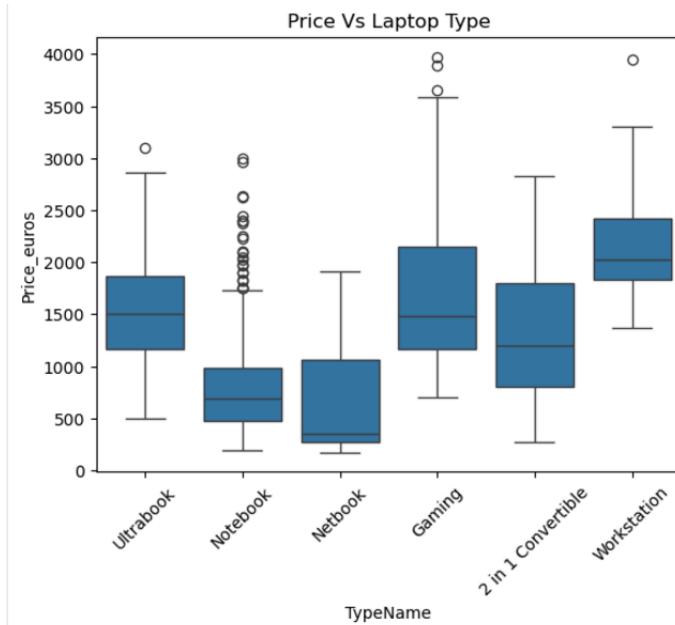


Figure 5.2.7: Price Distributions vs Screen Resolution

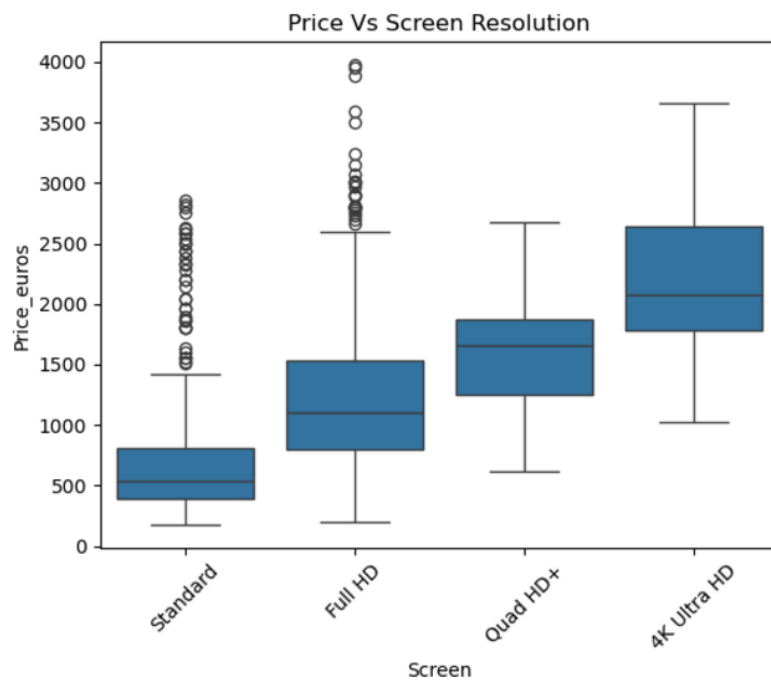


Figure 5.2.8: Dual Bar Chart Showing Primary vs Secondary Storage Type

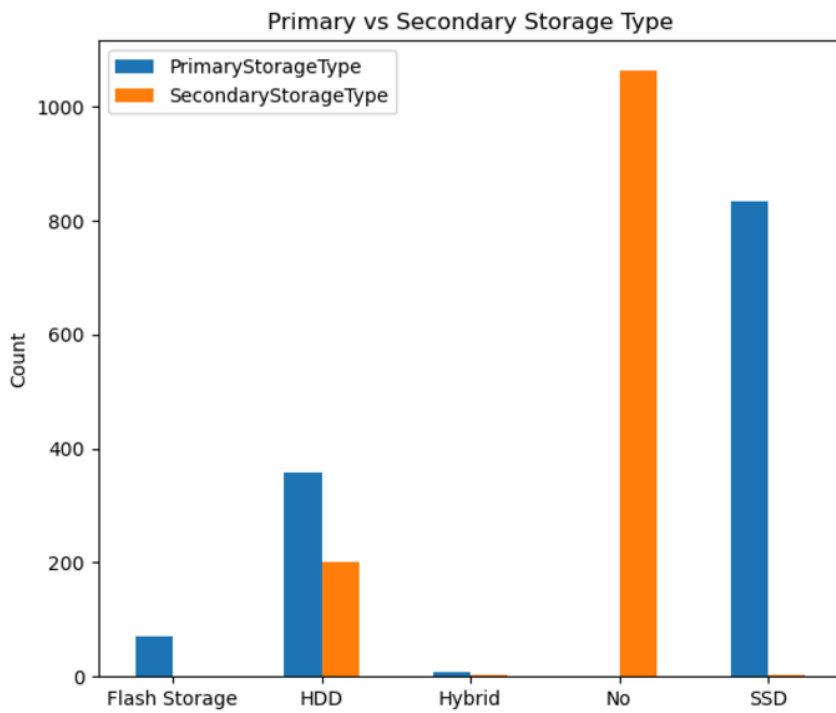


Figure 5.2.9: Dual Bar Chart Showing CPU and GPU

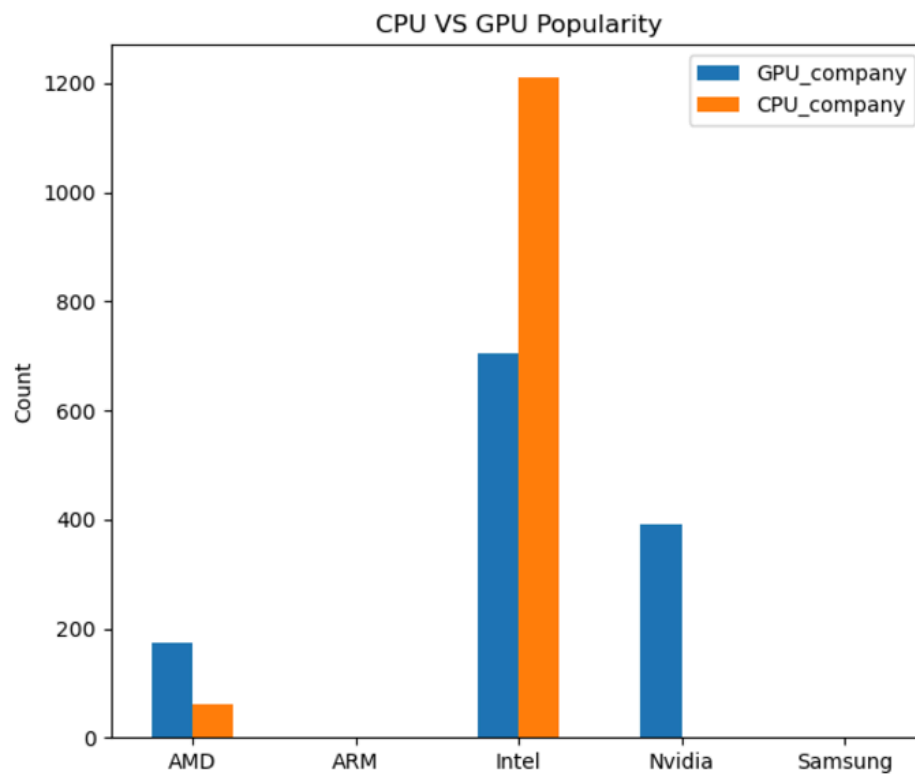


Figure 5.2.10: Box Plot Showing Price vs CPU

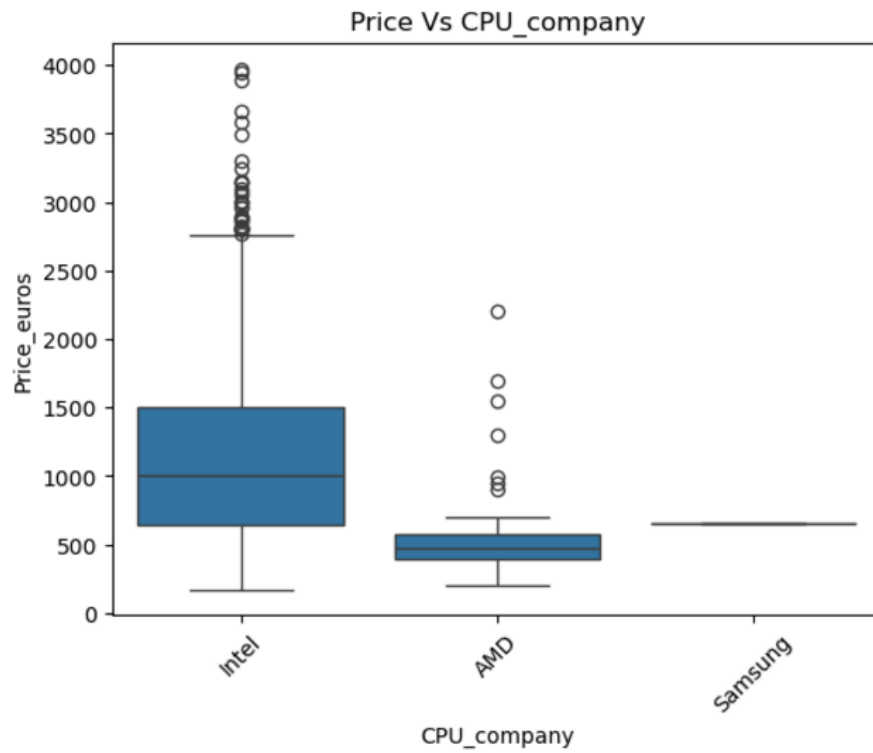
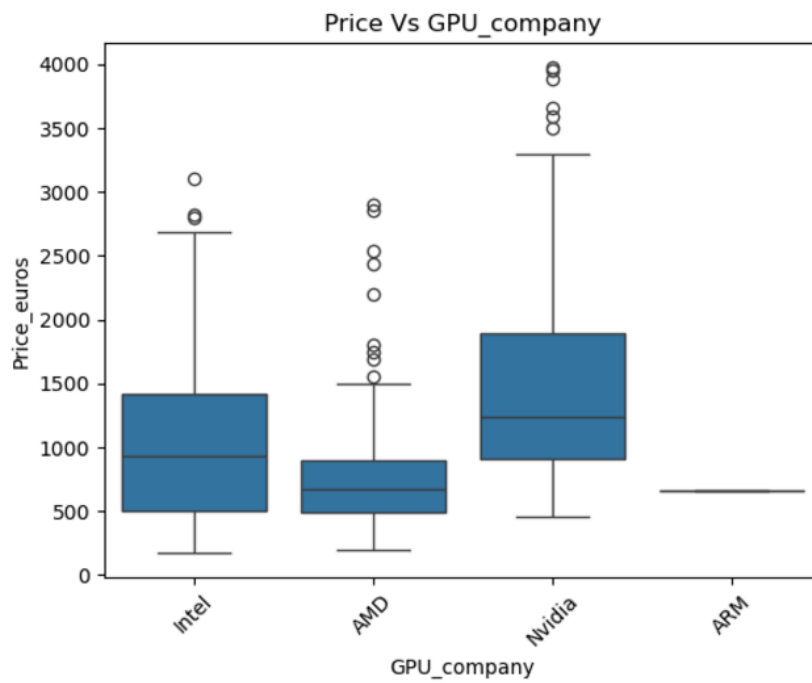


Figure 5.2.11: Box Plot Showing Price Vs GPU



Keyboards:

Figure 5.2.12: Swarmplot of Backlights Vs Price

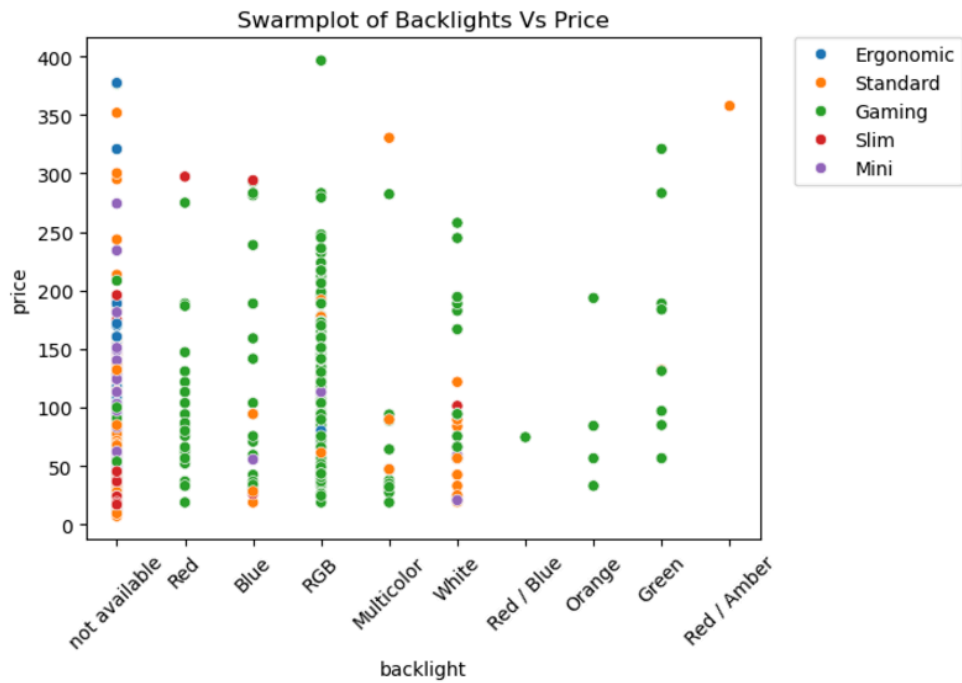
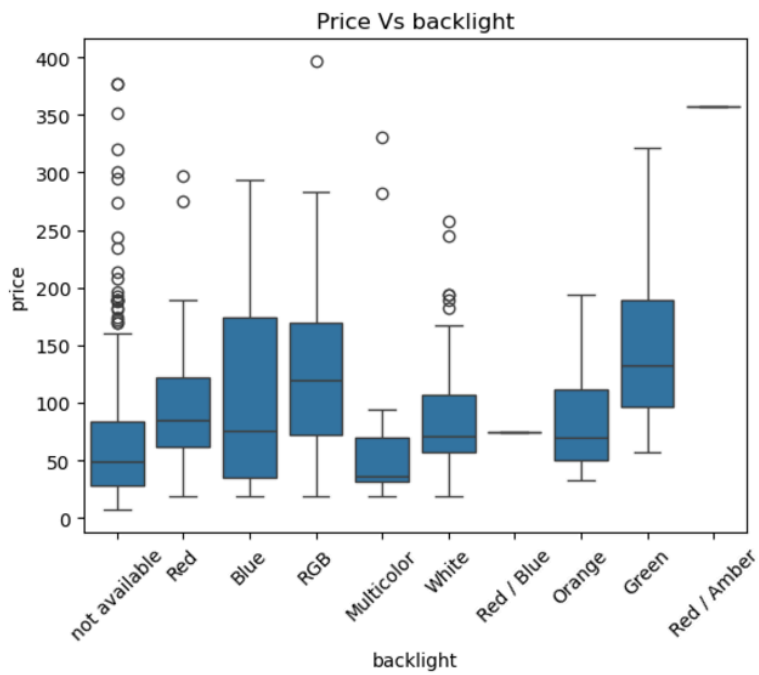


Figure 5.2.13 :Box Plot of Previous Figures



Appendix 2: Data Features

1. Laptop Prices Dataset Explanation

Feature Name	Type	Description
Company	Categorical (Nominal)	The name of the company responsible for manufacturing the PC (e.g. Dell, Apple).
Product	Categorical (Nominal)	The retail name of the computer (e.g. Macbook Air).
TypeName	Categorical (Nominal)	The purpose of the laptop (gaming, notebook, etc.).
Inches	Numeric (Continuous)	The width of the computer, in inches.
Ram	Numeric (Discrete)	The total Random Access Memory (RAM) of the computer, measured in GB.
OS	Categorical (Nominal)	Denotes the operating system (e.g. windows, linux).
Weight	Numeric (Continuous)	The overall weight of the computer, measured in KG.
Price_euros	Numeric (Continuous)	The price, in Euros, for the laptop in question.
Screen	Screen (Ordinal)	The definition of the display. Values include Standard, Full HD, 4K Ultra HD, Quad HD+ , uses an ordinal scale (lowest quality to highest quality).
ScreenW	Numeric (Continuous)	The width of the screen in pixels.
ScreenH	Numeric (Continuous)	The height of the screen in pixels.
Touchscreen	Categorical (Boolean)	Denotes the presence of touchscreen functionality (yes/no).
IPSPanel	Categorical (Boolean)	Denotes the presence of an IPSpanel Monitor (yes/no).
RetinaDisplay	Categorical	Denotes the presence of a RetinaDisplay (yes/no).

	(Boolean)	
CPU_company	Categorical (Nominal)	The company that manufactures the CPU.
CPU_freq	Numeric (Continuous)	The frequency of a laptop, also known as the clock speed, a measure of the number of cycles a CPU can perform per second, measured in hertz or gigahertz (Hawthorne, 2024). Wagih lists the feature as being measured in (Hz).
CPU_model	Categorical (Nominal)	The name of the CPU.
PrimaryStorage	Numeric (Discrete)	The total storage capacity, in GB, of the primary storage device listed on the computer.
SecondaryStorage	Numeric (Discrete)	The total storage capacity, in GB, of the secondary storage device listed on the computer.
PrimaryStorageType	Categorical (Discrete)	The type of storage used as primary storage (SSD, HDD, Flash Storage, or Hybrid)
SecondaryStorageType	Categorical (Nominal)	The type of storage used as primary storage (SSD, HDD, Flash Storage, Hybrid, or no, the latter indicating the laptop does not have a secondary storage unit).
GPU_company	Categorical (Nominal)	The manufacturer of the GPU (AMD, ARM, Intel, or Nvidia).
GPU_model	Categorical (Nominal)	The name of the GPU equipped to the device.

2. PCPartPicker API Dataset Explanation

CPU

Feature Name	Type	Description
brand	Categorical (Nominal)	The name of the company responsible for manufacturing the CPU.
model	Categorical (Nominal)	The retail name of the CPU.
cores	Numeric (Discrete)	The number of cores featured within a CPU, with more cores generally meaning higher performance.
base_clock_speed	Numeric (Discrete)	A measure of the number of cycles a CPU can perform per second, measured in hertz or gigahertz (Hawthorne, 2024).
tdp	Numeric (Discrete)	Thermal Design Power, used to measure the amount of heat being generated by the CPU (Edmonds, 2023).
integrated_graphics	Categorical (Nominal)	Refers to if a GPU is included as part of the CPU, or value None if the CPU is standalone.
multithreading	Categorical (Boolean)	Is multithreading possible?
price	Numeric (Continuous)	The price of the part in USD.

Headphones

Feature Name	Type	Description
brand	Categorical (Nominal)	The name of the company responsible for manufacturing the product.
model	Categorical (Nominal)	The retail name of the product.
form_factor	Categorical (Nominal)	Describes how the headphones interact with the ear (e.g circumaural headphones go over the user's head, while earbuds such as AirPods are placed inside the ear).

frequency_response	Numeric (Continuous)	Measures the range of frequencies that a set of headphones can deliver.
has_microphone	Categorical (Boolean)	Do the headphones have a built-in microphone (yes or no)?
is_wireless	Categorical (Boolean)	Do the headphones have a built-in microphone (yes or no)?
color	Categorical (Nominal)	The color of the product.
price	Numeric (Continuous)	The price of the product (USD).

Mice

Feature Name	Type	Description
brand	Categorical (Nominal)	The name of the company responsible for manufacturing the product.
model	Categorical (Nominal)	The retail name of the product.
tracking	Categorical (Nominal)	How the cursor moves across the screen when the mouse is moved.
connection	Categorical (Nominal)	How is the mouse connected to the computer, usually either as a wired connection or a bluetooth wireless connection, or sometimes the mouse is built into the laptop.
max_dpi	Numeric (Discrete)	Dots per inch, a measure of how sensitive the mouse is when moved.
hand_orientation	Categorical (Nominal)	Is the mouse designed to be used left-handed, right handed, or both.
color	Categorical (Nominal)	The color of the product.
price	Numeric (Continuous)	The price of the product (USD)

Keyboards

Feature Name	Type	Description
brand	Categorical (Nominal)	The name of the company responsible for manufacturing the product.
model	Categorical (Nominal)	The retail name of the product.
style	Categorical (Nominal)	The design purpose of the keyboard (e.g. gaming, Ergonomic, mini).
switches	Categorical (Nominal)	The type of key switches used in the keyboard.
backlight	Categorical (Nominal)	Reflects if the keyboard features built in lighting effects.
tenkeyless	Categorical (Boolean)	Does the keyboard have a 10-key numeric keypad (True or False)?
connection	Categorical (Nominal)	The method of how the keyboard connects to the computer.
color	Categorical (Nominal)	The color of the product.
price	Numeric (Continuous)	The price of the product (USD)

Appendix 3: Data Quality Issues and Remedies

Laptop Prices

Feature Name	Issue	Remedies
Company	None	None
Product	None	None
TypeName	None	None
Inches	None	None
Ram	None	None
OS	None	None
Weight	None	None
Price_euros	4 entries where price exists in the €4000-6000 range, over 3 standard deviations outside the mean.	Drop these rows, reducing the row count of the dataset from 1275 -> 1271.
Screen	None	None
ScreenW	None	None
ScreenH	None	None
Touchscreen	Stored as categorical feature, can be included as binary feature (true or false) to better enable one hot encoding in model building.	Converted feature values from “Yes” and “No” to True or False respectively.
IPSPanel	Stored as categorical feature, can be included as binary feature (true or false) to better enable one hot encoding in model building.	Converted feature values from “Yes” and “No” to True or False respectively.
RetinaDisplay	Stored as categorical feature, can be included as binary feature (true or false) to better enable one hot encoding in	Converted feature values from “Yes” and “No” to True or False respectively.

	model building.	
CPU_company	None	None
CPU_freq	None	None
CPU_model	None	None
PrimaryStorage	Treated SSD, HDD, Flash Storage as Identical when each storage type has differences not recorded in the dataset.	Replaced by features StorageSSD, StorageFlash, StorageHDD, StorageHybrid, and TotalStorage
SecondaryStorage	Treated SSD, HDD, Flash Storage as identical when each storage type has differences not recorded in the dataset.	Replaced by features StorageSSD, StorageFlash, StorageHDD, StorageHybrid, and TotalStorage
PrimaryStorageType	None	None
SecondaryStorageType	None	None
GPU_company	None	None
GPU_model	None	None

Laptop Prices (New Features)

Feature Name	Type	Description
StorageSSD	Numeric (Discrete)	The amount of SSD storage the device is capable of (GB).
StorageFlash	Numeric (Discrete)	The amount of flash storage the device is capable of (GB).
StorageHDD	Numeric (Discrete)	The amount of HDD storage the device is capable of (GB).
StorageHybrid	Numeric (Discrete)	The amount of Hybrid storage the device is capable of (GB).
TotalStorage	Numeric (Discrete)	Sum of both Primary and Secondary Storage

PCPartPicker API Dataset Explanation

Headphones

Feature Name	Issue	Remedies
brand	None	None
model	None	None
form_factor	None	None
frequency_response	1.Missing Values (23.9%) 2. Stored as FrequencyResponse class	1. Replaced with “not available”. 2. Converted to two new features, dfHeadphones["frequency_response_min"] and dfHeadphones["frequency_response_max"].
has_microphone	None	None
is_wireless	None	None
color	Missing Values (0.37%)	Replaced with “not available”.
price	1. Stored as a moneyed class. 2. Stored as USD while Laptops Dataset uses Euros. 3. Significant number of missing values (69.2%) 4. Significant outlier values (values over €500 when median price is €34.91)	1. Convert to Float 2. Convert to Euro at the rate of \$1 = €0.9438 3. Keep two versions of the dataset, the original version, and one where price = 0 (i.e. missing values). Use the original dataset for univariate analysis of categorical features, and the new dataset for multivariate analysis involving price. 4. Remove Outliers from the Dataset

Mice

Feature Name	Issue	Remedies
brand	None	None
model	None	None
form_factor	None	None
frequency_response	1. Missing Values (23.9%) 2. Stored as FrequencyResponse class	1. Replaced with “not available”. 2. Converted to two new features, <code>dfHeadphones["frequency_response_min"]</code> and <code>dfHeadphones["frequency_response_max"]</code> .
has_microphone	None	None
is_wireless	None	None
color	Missing Values (0.37%)	Replaced with “not available”.
price	1. Stored as a moneyed class. 2. Stored as USD while Laptops Dataset uses Euros. 3. Significant number of missing values (61%) 4. Significant outlier values (values over €200 when median price is €33)	1. Convert to Float 2. Convert to Euro at the rate of \$1 = €0.9438 3. Keep two versions of the dataset, the original version, and one where price = 0 (i.e. missing values). Use the original dataset for univariate analysis of categorical features, and the new dataset for multivariate analysis involving price. 4. Remove Outliers from the Dataset

Keyboards

Feature Name	Issue	Remedies
brand	None	None
model	None	None
style	None	None
switches	Missing Values	Replaced with “not available”.
backlight	Missing Values	Replaced with “not available”.
tenkeyless	None	None
connection	None	None
color	Missing Values	Replaced with “not available”.
price	1. Stored as a moneyed class. 2. Stored as USD while Laptops Dataset uses Euros. 3. Significant number of missing values (69%) 4. Significant outlier values	1. Convert to Float 2. Convert to Euro at the rate of \$1 = €0.9438 3. Keep two versions of the dataset, the original version, and one where price = 0 (i.e. missing values). Use the original dataset for univariate analysis of categorical features, and the new dataset for multivariate analysis involving price. 4. Remove Outliers from the Dataset

Appendix 4: Potential Price Gouging Example

I include this part here as although it is not possible to definitively prove price gouging in this assignment (and goes beyond the scope of the assignment) and takes a more anecdotal approach than an academic approach but I thought the topic was interesting and merited more discussion.

If we review the data points originally removed from the dataset, there were two Razer Blade Pro laptops that were listed as €6099 and €5,499 respectively both having a GTX1080 (see figure 9.1). However, when reviewing a reputable PC retailer (link [here](#)) we see that Blade Pro models with RTX30 & RTX40 graphics cards (a significantly more powerful model released by Nvidia) are available for considerably less (see figure 9.2).

Figure 9.1: Outlier Example

```
[49]: #Get Outliers  
df[df['Price_euros'] > 4000]
```

[49]:	Company	Product	TypeName	Inches	Ram	OS	Weight	Price_euros
196	Razer	Blade Pro	Gaming	17.3	32	Windows 10	3.49	6099.0
610	Lenovo	Thinkpad P51	Notebook	15.6	32	Windows 10	2.50	4899.0
749	HP	Zbook 17	Workstation	17.3	16	Windows 7	3.00	4389.0
830	Razer	Blade Pro	Gaming	17.3	32	Windows 10	3.49	5499.0

4 rows × 27 columns

Figure 9.2: Laptop from Link above:

Sort by
Popularity

Refine your results
Enter search term...

14 products




Razer Blade 18 Intel Core i9 32GB 1TB RTX 4090 240Hz QHD 18 Inch Windows 11 Gaming Laptop

SKU: RZ09-0484UWH4-R3W1

[Free Delivery](#) [Get Indiana Jones and the Great Circle™...](#)
[Get Assassins Creed Shadows»](#) [0% INTEREST FOR 12 MONTHS](#)

- Core i9 13950HX Processor
- GeForce RTX 4090 16GB Graphics card
- 18 Inch 2560 x 1600 Screen
- 32GB RAM
- 1TB SSD

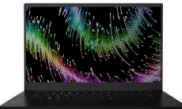
☐ Compare

SAVE €1844

€4391.97

[PayPal](#) Up to 12 Months 0% Credit

[View product](#)




Razer Blade 16 Intel Core i9 16GB 1TB RTX 4060 240Hz QHD+ 16 Inch Windows 11 Gaming Laptop

SKU: RZ09-0483RWH3-R3W1 [Read 6 reviews](#)

[Free Delivery](#) [Get Assassins Creed Shadows»](#) [Cyber Deal](#)

- Core i9 13950HX Processor
- GeForce RTX 4060 8GB Graphics card
- 16 Inch 2560 x 1600 Screen
- 16GB RAM
- 1TB SSD

☐ Compare

SAVE €1374

€2343.97

[View product](#)




Razer Blade 16 Intel Core i9 16GB 1TB RTX 4070 240Hz QHD 16 Inch Windows 11 Gaming Laptop

SKU: RZ09-0510SWN3-R3W1

[Free Delivery](#) [Get Indiana Jones and the Great Circle™...](#)
[Get Assassins Creed Shadows»](#)

- Core i9 14900HX Processor
- GeForce RTX 4070 8GB Graphics card

SAVE €817

€3129.97

Lexicon

PC: Personal Computer (note: this includes portable computers (Laptops) and stationary computers (Desktops))

GPU: Graphics Processor Unit

CPU: Central Processor Unit

GB: Gigabyte

SSD: Solid State Drive

HDD: Hard Disk Drives

RAM: Random Access Memory

OS: Operating System

For additional PC terminology, refer to the work of Corsair (2024)